Journal of Scientific Computing (2026) 107:87
https://doi.org/10.1007/s10915-026-03268-x

# Lift-and-Embed Learning Method for Solving Scalar Hyperbolic Equations with Discontinuous Solutions

**Qi Sun[1,2] · Zhenjiang Liu[1] · Lili Ju[3] · Xuejun Xu[1,2]**

Received: 18 March 2025 / Revised: 24 December 2025 / Accepted: 14 March 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

**Abstract**
Deep learning methods, which exploit auto-differentiation to compute derivatives without dispersion or dissipation errors, have recently emerged as a compelling alternative to classical mesh-based numerical schemes for solving hyperbolic conservation laws. However, solutions to hyperbolic problems are often piecewise smooth, posing challenges for training of neural networks to capture solution discontinuities and jumps across interfaces. In this paper, we propose a novel lift-and-embed learning method to effectively resolve these challenges. The proposed method comprises three innovative components: (i) embedding the Rankine-Hugoniot condition within a one-order higher-dimensional space by including an augmented variable; (ii) utilizing neural networks to handle the increased dimensionality and address both linear and nonlinear problems within a unified mesh-free learning framework; and (iii) projecting the trained model back onto the original physical domain to obtain the approximate solution. Notably, the location of discontinuities also can be treated as trainable parameters in our method and inferred concurrently with the training of neural network solutions. With collocation points sampled only on piecewise surfaces rather than fulfilling the whole lifted space, we demonstrate through extensive numerical experiments that our method can efficiently and accurately solve scalar hyperbolic equations with discontinuous solutions without spurious smearing or oscillations.

**Keywords** Hyperbolic partial differential equation · Linear and nonlinear flux model · Discontinuous solution · Deep learning · Neural network

✉ Qi Sun
qsun_irl@tongji.edu.cn

Zhenjiang Liu
2333705@tongji.edu.cn

Lili Ju
ju@math.sc.edu

Xuejun Xu
xuxj@tongji.edu.cn

[1] School of Mathematical Sciences, Tongji University, Shanghai 200092, China

[2] Key Laboratory of Intelligent Computing and Applications (Ministry of Education), Tongji University, Shanghai 200092, China

[3] Department of Mathematics, University of South Carolina, Columbia, SC 29208, USA

Published online: 04 May 2026

**Mathematics Subject Classification** 35L60 · 35L67

## 1 Introduction

The modeling of various problems in natural sciences and engineering is rooted in the concept of conservation laws, e.g., the Euler equations in fluid dynamics [43], the Lighthill-Whitham-Richards equations for traffic flow [21], and Maxwell's equations of electromagnetics [32], all of which are mathematically described as hyperbolic partial differential equations [23]. However, numerical simulation of these systems remains challenging, primarily due to discontinuities induced by non-smooth initial data or intersecting characteristics. Traditional numerical methods using uniform meshes, such as the upwind and Lax-Wendroff schemes [30], suffer from dispersion or dissipation issues near discontinuities. A variety of refinements, including but not limited to ENO/WENO-based finite volume schemes [31, 32, 43, 60], discontinuous/adaptive finite element methods [13, 16, 20, 28], have achieved broad success in capturing sharp solution transitions. However, anomalous solutions, such as the carbuncle phenomenon [40, 56], are observed when using flux limiters within high-resolution methods, necessitating specialized adjustments through ad hoc stabilization techniques. In addition to the escalating algorithmic complexity, accurately resolving discontinuities often requires high-quality meshes and anisotropic refinements, the generation of which could be time-consuming for complex interfaces and high-dimensional problems.

With the rapid advancement of hardware and software resources, artificial neural network-based deep learning methods have emerged as a prominent approach in contemporary scientific computing, particularly valued for their universal approximation capability [33], flexible meshless implementation [57], and effectiveness in addressing the curse of dimensionality [3]. One representative work is the physics-informed neural networks (PINNs) [37, 42, 57], which trains network models to minimize loss functions derived from the residual of differential equations at selected collocation points. Unlike numerical differentiation employed in solving hyperbolic equations [28], auto-differentiation within neural networks [54] enables precise derivative evaluation and has attracted significant attention for its ability to mitigate stability issues [1, 48, 55, 63]. Unfortunately, standard network architectures often struggle to capture solution discontinuities and jumps arising from shock waves or contact discontinuities [25, 35, 55, 65], spurring ongoing efforts to enhance their ability for resolving discontinuities.

For instance, discontinuous solutions can be artificially smoothed by incorporating an artificial viscosity term [55] into the hyperbolic system, but it would incur substantial costs due to the necessity of computing second-order derivatives through automatic differentiation. To maintain the simplicity of the equations, adaptive activation functions [19, 36] have been proposed to facilitate the use of non-differentiable activation functions for shock capturing, while another line of methods resort to the adaptive sampling of collocation points near regions of discontinuity [27, 51]. Recent studies suggest that the primary challenge in addressing discontinuous solutions lies in their rigorous mathematical definition as weak solutions [48], which motivates the design of loss functions based on control volume formulations [55], variational formulations [18], and some of their variants [46]. To further enhance computational efficiency, domain decomposition strategies are integrated into the training of local network solutions as well as the interfaces that separate them [48]. Meanwhile, considerable efforts have been devoted to combining traditional numerical methods with deep learning techniques [2, 12, 24, 53], aiming to harness the strengths of both paradigms.

In contrast to prior methods, in this work we introduce a novel lift-and-embed learning framework for solving scalar hyperbolic equations with discontinuous solutions. First, the Rankine-Hugoniot relation [43], derived from the local weak form of conservation laws across discontinuities, is embedded in a one-order higher-dimensional space by augmenting the solution representation with an auxiliary variable. Notably, the augmented variable endows our solution ansatz with a piecewise smoothness property, thereby eliminating the need for multi-valued function approximations and enabling neural networks to easily reconstruct sharp discontinuities. Besides, our reformulated equations are defined on piecewise surfaces rather than being distributed over the entire lifted space, ensuring the generation process of collocation points remains unaffected despite the increased dimensionality. Second, neural networks are utilized to parametrize the unknown solution, which alleviate the curse of dimensionality and serve as universal function approximators for addressing both linear and nonlinear problems. Upon the completion of training process [37], the approximate solution is obtained by projecting the trained network solution back onto the original physical domain. In particular, building on deep learning advances in inverse problems [57], we are also able to incorporate unknown discontinuity locations as additional trainable parameters, which then can be learned jointly with the neural network solution. Finally, experimental studies are conducted on a wide range of scalar hyperbolic benchmarks, as well as the Euler system governing the Sod shock tube problem, wherein all discontinuities are well caught without the manifestation of spurious numerical smearing or oscillations.

The remainder of this work is organized as follows. A brief introduction to hyperbolic equations is first provided in Section 1.1. Section 2 presents relevant literature on existing traditional and deep learning-based numerical methods for hyperbolic equations, followed by an exploration of our motivations from the perspective of function approximation. Section 3 is devoted to detailing our lifted-and-embed learning method and corresponding algorithms with identified or unknown discontinuity locations, supplemented by two benchmark problems to enhance clarity. Next, numerical experiments conducted on various problems are reported in Section 4, demonstrating the effectiveness and efficiency of our proposed methods in handling discontinuous solutions. Finally, a concluding remark and future work are given in Section 5.

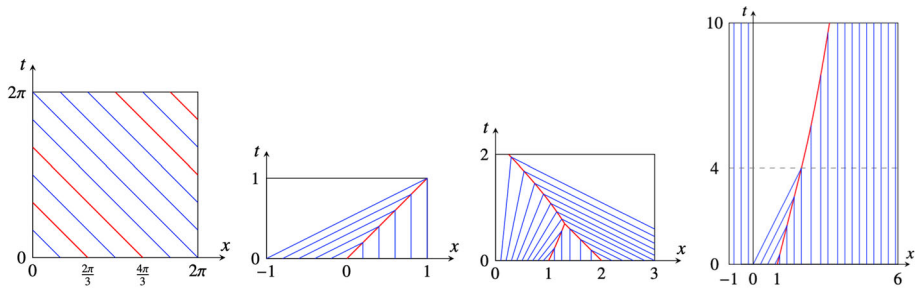
### 1.1 Hyperbolic Conservation Laws

Scalar conservation laws arise from various physical and engineering disciplines, which are typically formulated as hyperbolic equations of the following form [23]

$$\begin{cases} \partial_t u(x,t) + \nabla_x \cdot f(u(x,t)) = 0, & \text{in } \Omega \times (0,T], \\ u(x,0) = u_0(x), & \text{on } \Omega, \\ u(x,t) = g(x,t), & \text{on } \Sigma \times (0,T], \end{cases} \tag{1}$$

where $\Omega \subset \mathbb{R}^d$ ($d \in \mathbb{N}_+$) is an open bounded domain with Lipschitz boundary $\partial\Omega$, $[0,T]$ the temporal interval, $u(x,t) \in \mathbb{R}^1$ a conserved quantity, $f(u) = (f_1(u), \cdots, f_d(u))$ the outward spatial flux vector, $\nabla_x \cdot$ a divergence operator with respect to the $x$-variable, $u_0(x)$ the initial data, and $g(x,t)$ the inflow data defined on parts of the boundary $\Sigma \subset \partial\Omega$ where characteristics enter the computational domain. Notably, classical solutions to the problem (1) may not exist due to discontinuous initial data or intersecting characteristics (see Figure 1), thereby necessitating the introduction of weak solutions [23, 28].

The concept of weak derivatives enables the analysis of solutions to the problem (1) that exhibit only piecewise smoothness. Given an orientable surface of discontinuity $\Gamma$, we




(a) Convection equation (4) (b) Burgers' equation (9) (c) Burgers' equation (16) (d) Burgers' equation (17)

**Fig. 1** Characteristic and discontinuity curves (marked in blue and red colors, respectively) for linear and quasi-linear hyperbolic equations

denote by $\boldsymbol{n} = (n_t, n_{x_1}, \cdots, n_{x_d})^T \neq (0, 0, \cdots, 0)^T$ a normal vector to it and by $u^+(x,t)$ and $u^-(x,t)$ the limits of $u(x,t)$ on each side of $\Gamma$, namely, for any $\varepsilon > 0$,

$$u^{\pm}(x,t) = \lim_{\varepsilon \to 0} u((x,t) \pm \varepsilon \boldsymbol{n}).$$

Note that if $(n_{x_1}, n_{x_2}, \cdots, n_{x_d}) \neq (0, 0, \cdots, 0)$, the normal vector can be rewritten as $\boldsymbol{n} = (-s, \boldsymbol{v})^T$ where $s \in \mathbb{R}$ and $\boldsymbol{v} = (v_1, v_2, \cdots, v_d)$ is a unit vector in $\mathbb{R}^d$ [28]. By denoting $[\![\cdot]\!]$ the difference of quantity across the discontinuity surface, i.e., $[\![u]\!] = u^+ - u^-$ and $[\![f(u)]\!] = f(u^+) - f(u^-)$, the Rankine-Hugoniot relation is introduced below to ensure the consistency of conservation laws across discontinuities [28].

**Lemma 1** [28] *Let $u \in L^\infty(\Omega \times [0,T])$ be a piecewise $C^1$ function, then $u(x,t)$ is a weak solution of (1) if and only if the two following conditions are satisfied:*

(1) *$u(x,t)$ is a classical solution of (1) in the domains where $u(x,t)$ is $C^1$;*
(2) *$u(x,t)$ satisfies the jump condition along the surfaces of discontinuity, i.e.,*

$$s[\![u]\!] = \sum_{j=1}^{d} v_j [\![f_j(u)]\!] \tag{2}$$

*where $s$ can be interpreted as the speed of propagation of the discontinuity.*

***Remark 1*** For a smooth discontinuity curve $\Gamma = (t, \gamma(t))$ in one-dimension, we have $\boldsymbol{v} = 1$ and $s(t) = \frac{d\gamma(t)}{dt}$.

## 2 Related Work and Motivations

### 2.1 Related Work

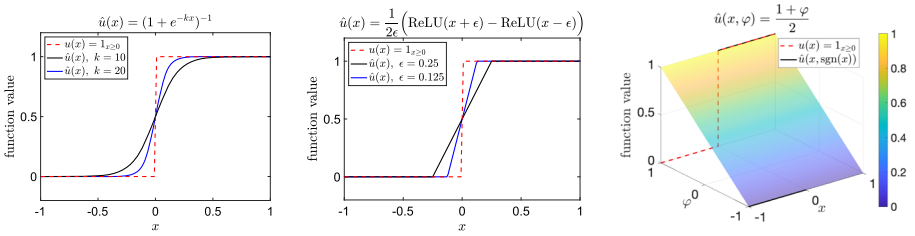
In contrast to the traditional mesh-based discretization of differential operators [28], deep learning methods leverage automatic differentiation to compute derivatives [42, 57], providing a mesh-free alternative that can circumvent dispersion and dissipation issues encountered in the numerical simulation of hyperbolic problems. However, solutions to hyperbolic equations usually develop discontinuities over time and neural network methods often struggle

to capture such solution jumps, necessitating specialized techniques to augment existing learning approaches [37] for effective discontinuity resolution.

Rather than relying on standard network architectures, adaptive activation functions [19, 36] and attention-based neural networks [58] have been proposed to enhance the representation of discontinuous solutions, paired with adaptive sampling strategies [26, 27, 47, 51] that prioritize training data allocation near regions of discontinuity. Nevertheless, as reported in [48], standard neural networks remain effective for computing rarefaction waves that constitute continuous solutions. On the other hand, recent studies suggest that the primary challenge in addressing discontinuous solutions arises from their mathematical characterization as weak solutions, hence independently of the regularity of the activation function being implemented [48]. Motivated by the method of vanishing viscosity [23], a widely adopted approach involves introducing an artificial diffusion term to mitigate the problem's hyperbolicity [14, 55]. However, determining the proper amount and location of artificial viscosity is technical: insufficient viscosity leads to numerical stiffness, whereas excessive viscosity over-smoothes the solution. The homotopy dynamics method [11] addresses this challenge by gradually training the neural network solution from a tractable high-viscosity problem to the target low-viscosity problem. While these techniques regularize the solution near discontinuities and enables seamless auto-differentiation across the entire computational domain, they convert the original problem into a parabolic type, which can often result in a degradation in the solution accuracy. Additionally, computing second-order derivatives can significantly slow down the training process, as each spatial point requires two rounds of backpropagation instead of a single backward pass for constructing the residual loss function [22].

Recall that a key feature of hyperbolic problems lies in their tendency to develop discontinuous solutions within a finite time, which indicates that the underlying equations should be considered in a weak sense rather than in a pointwise (strong) manner. Accordingly, the weak formulation of hyperbolic equations is recast as a min-max optimization problem [18], where the network solution is maximized with respect to test functions and minimized with respect to trial functions. This approach simplifies to a minimization problem when adopting a mesh-based representation for test functions [38], though it may compromise the meshless advantage offered by neural networks [18]. Alternatively, the weak formulation can be characterized locally by the Rankine-Hugoniot condition [43], which allows for the design of adaptive weights that can autonomously switch between strong and weak formulation-based loss functions [46]. Moreover, the computation of model solutions could be parallelized using domain decomposition strategies [48], with discontinuity interfaces co-evolved during training.

By partitioning the space-time domain into disjoint subregions and applying the Gauss divergence theorem [23] to each, hyperbolic equations can also be expressed in an integral form, referred to as the control volume formulation [43], which admits weak solutions involving discontinuities and therefore facilitates the development of various learning algorithms [7–10, 55]. Concurrently, much effort has been made to incorporating deep learning techniques into existing numerical methods, such as employing neural networks to learn, either partially or entirely, the finite volume [2, 4, 53] or discontinuous Galerkin schemes [12] with specialized numerical flux functions. Encoding the distinctive structure of hyperbolic equations [52, 68], such as characteristic curves [5] or implicit forms [67], within the network architecture offers another way of integrating prior domain knowledge.




(a) Smooth approximation (b) Piecewise approximation (c) Lift-and-embed reconstruction

**Fig. 2** Approaches for approximating the Heaviside function: (**a**) a kernel smoothing, (**b**) a finite element interpolation, (**c**) a lift-and-embed technique

## 2.2 Motivations from A Function Approximation Perspective

To elucidate our motivations in handling discontinuities, we present an illustrative example involving the analytic approximation of Heaviside step function $u(x) = H(x) := \mathbb{1}_{x\geq 0}$ using kernel smoothing, finite element interpolation, and lift-and-embed technique, as depicted in Figure 2. More precisely, a common approach is to apply the convolution operation with suitable kernel functions, e.g.,

$$u(x) \approx \hat{u}(x) := (u * G)(x) = \int_{-\infty}^{+\infty} u(\tau)G(x-\tau)\,d\tau = \frac{1}{1+e^{-kx}},$$

where $G(y) = ke^{-ky}(1+e^{-ky})^{-2}$ is the Sigmoid kernel function. As is well known, a larger value of parameter $k > 0$ corresponds to a sharper transition at $x = 0$ (see Figure 2a), in which the discontinuity is smoothed out in a manner analogous to the method of vanishing viscosity applied to hyperbolic equations [55].

Alternatively, consider the partition $-1 = x_0 < x_1 < x_2 < x_3 = 1$ of interval $[-1, 1]$ with $x_2 = -x_1 = \epsilon$, another widely used approach is to approximate the target function using piecewise linear elements (see Figure 2b), i.e.,

$$u(x) \approx \hat{u}(x) := \sum_{j=0}^{3} u(x_j)\phi_j(x) = \frac{1}{2\epsilon}\Big(\mathrm{ReLU}(x+\epsilon) - \mathrm{ReLU}(x-\epsilon)\Big),$$

where basis functions $\{\phi_j(x)\}_{j=0}^3$ are rewritten in terms of the rectified linear unit function $\mathrm{ReLU}(x) = \max(0, x)$ [66]. The latter expression is also known as a free-knot spline with two knots or as a single-hidden-layer feedforward neural network [9].

Beyond these two representative methods, discontinuous functions can be embedded in a one-order higher-dimensional space to achieve continuity [34, 64]. The key idea involves including an augmented variable to allocate the jump in $u(x) = \mathbb{1}_{x\geq 0}$ to separate branches, and a concrete example is displayed in Figure 2c, namely,

$$u(x) = \hat{u}(x, \mathrm{sgn}(x)) \quad \text{with} \quad \hat{u}(x, \varphi) := \frac{1-\varphi}{2}u^- + \frac{1+\varphi}{2}u^+ = \frac{1+\varphi}{2},$$

where $u^- = 0$ and $u^+ = 1$. Clearly, function $\hat{u}(x, \varphi)$ is no longer discontinuous with respect to the $x$-variable and admits exact recovery by choosing $\varphi = \mathrm{sgn}(x) = 2(\mathbb{1}_{x\geq 0} - 0.5)$. While the lifting operation often yields a high-dimensional function that may incur the curse of dimensionality, neural networks are highly effective at addressing this issue [3]. Moreover, discontinuity locations can be regarded as unknown parameters to be inferred during training [48], which deep learning-based methods are proven to be well-suited to resolve [57].

## 3 The Proposed Lift-and-Embed Learning Method for Scalar Hyperbolic Equations

In this section, we present a lift-and-embed learning method for solving scalar hyperbolic equations with discontinuous solutions, addressing scenarios with and without a-priori knowledge of discontinuity locations. We begin by embedding the target equations into a one-order higher-dimensional space, illustrated with two benchmark examples to show the effectiveness in resolving discontinuities while maintaining overall continuity. Next, detailed algorithms are presented, as well as the parametrization of discontinuity interfaces as trainable parameters, which allows us to harness the flexibility of the proposed deep learning method in solving both forward problems and parameter inversion.

### 3.1 Hyperbolic Equations Embedded in A One-order Higher-Dimensional Space

Here, we focus on a single discontinuity interface that divides the space-time domain into two subregions, which can be generalized to scenarios containing multiple interfaces. Based on the parametrization of interface

$$\Gamma = \Gamma(t, x_1, x_2, \cdots, x_d) \quad \text{or} \quad x_d = \gamma(t, x_1, \cdots, x_{d-1}),$$

we construct the solution ansatz to (1) by incorporating an augmented variable

$$u(x,t) = \hat{u}(x,t,\varphi(x,t)) \quad \text{with} \quad \varphi(x,t) = H(x_d - \gamma(t, x_1, \cdots, x_{d-1})),$$

where $H(x) = \mathbb{1}_{x\geq 0}$ is the Heaviside step function. Though the inclusion of an extra augmented variable appears to increase the dimensionality, its dependence on intrinsic variables $(x,t)$ remains unchanged.

Let $\delta(*)$ denote the Dirac delta function, a direct calculation implies that

$$\begin{aligned}
\partial_t u &= \partial_t \hat{u} - \delta(x_d - \gamma)\, \partial_\varphi \hat{u}\, \partial_t \gamma, \\
\nabla_x \cdot f(u) &= f_d'(\hat{u})(\partial_{x_d}\hat{u} + \delta(x_d - \gamma)\, \partial_\varphi \hat{u}) + \sum_{j=1}^{d-1} f_j'(\hat{u})(\partial_{x_j}\hat{u} - \delta(x_d - \gamma)\, \partial_\varphi \hat{u}\, \partial_{x_j}\gamma),
\end{aligned}$$

while the Rankine-Hugoniot jump condition (2) can be reformulated as

$$s\big(\hat{u}(x,t,\varphi^+(x,t)) - \hat{u}(x,t,\varphi^-(x,t))\big) = \sum_{j=1}^{d} \nu_j \big(f_j\big(\hat{u}(x,t,\varphi^+(x,t))\big) - f_j\big(\hat{u}(x,t,\varphi^-(x,t))\big)\big)$$

valong the surface of discontinuity. Then, the problem (1) is embedded into a one-order higher-dimensional space as follows:

$$\begin{cases}
\partial_t \hat{u}(x,t,\varphi(x,t)) + \nabla_x \cdot f(\hat{u}(x,t,\varphi(x,t))) = 0, & \text{for } (x,t) \in \Omega \times (0,T] \setminus \Gamma, \\
s[\![\hat{u}(x,t,\varphi(x,t))]\!] = \sum_{j=1}^{d} \nu_j [\![f_j(\hat{u}(x,t,\varphi(x,t)))]\!], & \text{for } (x,t) \in \Gamma, \\
\hat{u}(x,0,\varphi(x,0)) = u_0(x), & \text{for } (x,t) \in \Omega \times \{0\}, \\
\hat{u}(x,t,\varphi(x,t)) = g(x,t), & \text{for } (x,t) \in \Sigma \times (0,T],
\end{cases} \tag{3}$$

in which the solution jump is imposed on separate branches that greatly differs from its lower-dimensional counterpart (2). Before detailing our learning algorithm, two benchmark

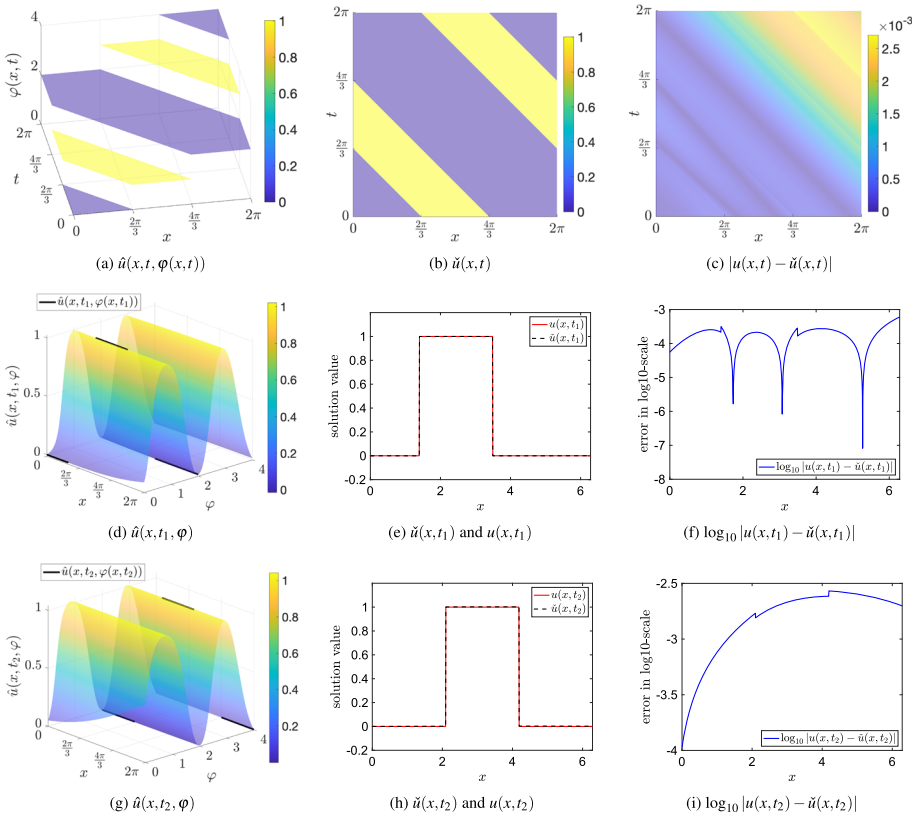



**Fig. 3** Numerical results for the linear convection equation (4) ($t_1 = \frac{2\pi}{9}$, $t_2 = 2\pi$)

examples with analytically defined interfaces of discontinuity are provided to illustrate the effectiveness of equations (3) in handling discontinuous solutions.

### 3.1.1 Example 1: A Linear Convection Equation in One-Dimension

Let us consider the following one-dimensional convection equation [30]:

$$\begin{cases} \partial_t u(x,t) - \partial_x u(x,t) = 0, & \text{for } (x,t) \in \Omega = (0,2\pi)\times(0,2\pi], \\ u_0(x) = H(x-\frac{2\pi}{3}) - H(x-\frac{4\pi}{3}), & \text{for } x \in (0,2\pi), \\ u(0,t) = u(2\pi,t), & \text{for } t \in (0,2\pi], \end{cases} \tag{4}$$

where the exact solution $u(x,t) = u_0(x+t)$. Notably, the above initial data is given by a square wave, namely,

$$u_0(x) = \begin{cases} 0, & \text{for } 0 \le x < \frac{2\pi}{3}, \\ 1, & \text{for } \frac{2}{3}\pi \le x < \frac{4\pi}{3}, \\ 0, & \text{for } \frac{4}{3}\pi \le x \le 2\pi, \end{cases} \tag{5}$$

where traditional mesh-based numerical schemes [30] often encounter issues with dissipation or dispersion near discontinuities as the solution evolves over time.

Note that, by Lemma 1, the propagation speed of discontinuities for the problem (4)

$$s = \frac{[\![ f(u) ]\!]}{[\![ u ]\!]} = -1 \quad \text{where} \quad f(u) = -u \quad \text{and} \quad \boldsymbol{\nu} = 1,$$

is the same as the slope of characteristic lines (see Figure 1a). The augmented variable can then be constructed to embed the problem (4) into a one-order higher-dimensional space (see Figure 3a) as follows:

$$u(x,t) = \hat{u}(x,t,\varphi(x,t)) \quad \text{with} \quad \varphi(x,t) = \sum_{i=1}^{2}\sum_{k=0}^{n_i} H(x - st - x_i - 2k\pi), \tag{6}$$

where $x_1 = \frac{2\pi}{3}$, $x_2 = \frac{4\pi}{3}$, $T = 2\pi$, and $n_i = \lceil -\frac{sT+x_i}{2\pi} \rceil$ for $s < 0$ and $i = 1, 2$. Moreover, the solution jump arising from the initial data (5) implies that,

$$[\![ u_0(x_i) ]\!] = [\![ \hat{u}(x,t,\varphi(x,t)) ]\!] = \hat{u}(x,t,\varphi^+(x,t)) - \hat{u}(x,t,\varphi^-(x,t)) \text{ on } \Gamma_i \tag{7}$$

where $\Gamma_i = \{(x(t), t) \mid x(t) = st + x_i + 2k\pi\}$ for $0 \le k \le n_i$ and $1 \le i \le 2$.

It is noteworthy that the right and left limits of our augmented variable $\varphi(x,t)$ at $\Gamma = \Gamma_1 \cup \Gamma_2$ can take distinct values in (7), allowing the jump of solution $u(x,t)$ across the discontinuity interface to be exactly captured by the difference of ansatz $\hat{u}(x,t,\varphi(x,t))$ at separate points. On the other hand, a direct computation implies

$$\partial_t u = \partial_t \hat{u} - s\partial_\varphi \hat{u} \sum_{i=1}^{2}\sum_{k=0}^{n_i} \delta(x - st - x_i - 2k\pi), \; \partial_x u = \partial_x \hat{u} + \partial_\varphi \hat{u} \sum_{i=1}^{2}\sum_{k=0}^{n_i} \delta(x - st - x_i - 2k\pi).$$

To sum up, the hyperbolic system satisfied by our solution ansatz $\hat{u}(x,t,\varphi(x,t))$ takes on the form

$$\begin{cases} \partial_t \hat{u}(x,t,\varphi(x,t)) - \partial_x \hat{u}(x,t,\varphi(x,t)) = 0, & \text{for } (x,t) \in \Omega \setminus \Gamma_1 \cup \Gamma_2, \\ [\![ \hat{u}(x,t,\varphi(x,t)) ]\!] = [\![ u_0(x_i) ]\!], & \text{for } (x,t) \in \Gamma_i \ (i = 1, 2), \\ \hat{u}(x,0,\varphi(x,0)) = u_0(x), & \text{for } x \in (0, 2\pi), \\ \hat{u}(0,t,\varphi(x,0)) = \hat{u}(2\pi,t,\varphi(2\pi,t)), & \text{for } t \in (0, 2\pi], \end{cases} \tag{8}$$

which enables a smooth representation of the discontinuous solution problem (4) by bridging the jump conditions (7) within a one-order higher-dimensional space (see (d) and (g) in Figure 3). The solution ansatz is then parametrized using a fully-connected neural network [29], followed by a vanilla training process under the pointwise residual-minimization framework [37, 57]. The trained neural network solution (still denoted as $\hat{u}(x,t,\varphi(x,t))$) and its projection back to the lower-dimensional space (denoted as $\breve{u}(x,t)$) are shown in (b), (e), and (h) of Figure 3, together with their error profiles without notable dissipation or dispersion errors (see (c), (f), and (i) in Figure 3).

### 3.1.2 Example 2: An Inviscid Burgers' Equation in One-Dimension

Next, we consider the following one-dimensional inviscid Burgers' equation [30, 43]:

$$\begin{cases} \partial_t u(x,t) + u(x,t)\partial_x u(x,t) = 0, & \text{for } (x,t) \in \Omega = (-1,1) \times (0,1], \\ u_0(x) = 2H(-x), & \text{for } x \in (-1,1), \\ u(-1,t) = 2, \ u(1,t) = 0, & \text{for } t \in (0,1]. \end{cases} \tag{9}$$

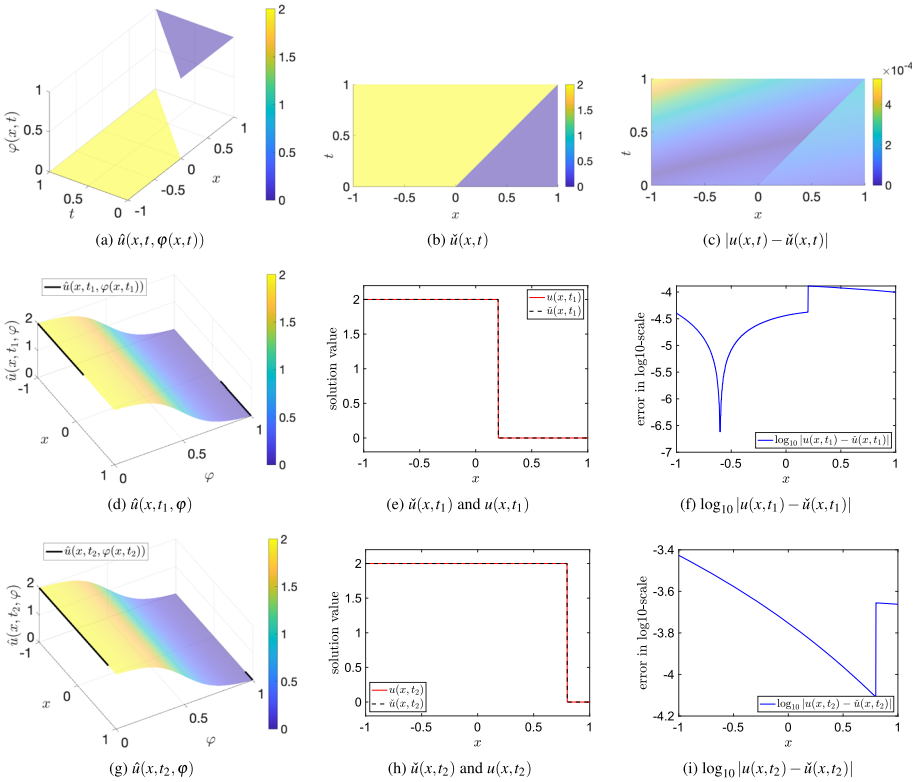



**Fig. 4** Numerical results for the inviscid Burgers' equation (9) ($t_1 = 0.2$, $t_2 = 0.8$)

In contrast to the linear equations (4), the characteristics of problem (9) now cross each other as shown in Figure 1b, leading to the formation of shock discontinuities at intersection points.

Based on the Rankine-Hugoniot relation, the shock speed is a constant, i.e.,

$$s = \frac{[\![f(u)]\!]}{[\![u]\!]} = \frac{1}{2}\left(u^+(x,t) + u^-(x,t)\right) = 1 \quad \text{where} \quad f(u) = \frac{1}{2}u^2 \quad \text{and} \quad \boldsymbol{v} = 1,$$

and therefore the shock curve is a straight line departing from the origin. Then, our solution ansatz is defined as

$$u(x,t) = \hat{u}(x,t,\varphi(x,t)) \quad \text{with} \quad \varphi(x,t) = H(x - st), \tag{10}$$

which also allows for the reformulation of Rankine-Hugoniot jump condition as

$$s = \frac{[\![f(\hat{u})]\!]}{[\![\hat{u}]\!]} = \frac{1}{2}\left(\hat{u}(x,t,\varphi^+(x,t)) + \hat{u}(x,t,\varphi^-(x,t))\right) \quad \text{on} \quad \Gamma = \left\{(x(t),t) \;\middle|\; x(t) = st\right\}.$$

In other words, the difference in values of $\hat{u}(x,t,\varphi(x,t))$ on opposing sides of the discontinuity interface $\Gamma$ corresponds to the jump of original solution $u(x,t)$ along the shock curve (see Figure 4a for its numerical realization).

On the other hand, a direct computation implies that

$$\partial_t u = \partial_t \hat{u} - s\partial_\varphi \hat{u}\delta(x - st) \quad \text{and} \quad \partial_x u = \partial_x \hat{u} + \partial_\varphi \hat{u}\delta(x - st),$$

and therefore the hyperbolic system satisfied by our solution ansatz $\hat{u}(x,t,\varphi(x,t))$ is given by

$$\begin{cases} \partial_t \hat{u}(x,t,\varphi(x,t)) - \hat{u}(x,t,\varphi(x,t))\partial_x \hat{u}(x,t,\varphi(x,t)) = 0, & \text{for } (x,t) \in \Omega \setminus \Gamma, \\ \hat{u}(x,t,\varphi^+(x,t)) + \hat{u}(x,t,\varphi^-(x,t)) = 2, & \text{for } (x,t) \in \Gamma, \\ \hat{u}(x,0,\varphi(x,0)) = u_0(x), & \text{for } x \in (-1,1), \\ \hat{u}(-1,t,\varphi(-1,t)) = 2,\ \hat{u}(1,t,\varphi(1,t)) = 0, & \text{for } t \in (0,1], \end{cases} \tag{11}$$

which offers a smooth representation of the shock wave problem (9) within a one-order higher-dimensional space. To address the computational challenge posed by the increased dimensionality, a fully-connected neural network [29] is deployed to parametrize our solution ansatz, followed by a standard training procedure [37, 57] as before.

The trained neural network solution and its projection back to the $(x,t)$-plane, denoted as $\hat{u}(x,t,\varphi(x,t))$ and $\breve{u}(x,t)$ respectively, are shown in Figure 4, as well as their error profiles. Remarkably, the approximation accuracy for inviscid Burgers' equation (11) is comparable to that of the convection problem (8), which showcases the flexibility of our learning methods in addressing both linear and nonlinear problems. Moreover, the inclusion of augmented variable facilitates the representation of discontinuous solutions on a smooth surface (see (d) and (g) in Figure 4), therefore allowing for a satisfactory reconstruction of discontinuities without laborious adjustment of sampling strategies or penalty viscous terms.

### 3.2 The Lift-and-Embed Learning Method

Thanks to the lift-and-embed operation, a fully connected neural network [29] with smooth activation functions, e.g., $\sigma(x) = \tanh(x)$, can be naturally applied to parametrize our solution ansatz (not relabelled), i.e.,

$$\hat{u}(x,t,\varphi;\theta) = (L_{D+1} \circ \sigma \circ L_D \circ \cdots \circ \sigma \circ L_1 \circ L_0)(x,t,\varphi)$$

where $D \in \mathbb{N}_+$ denotes the number of hidden layers and $\theta$ indicates the collection of model parameters. It is also noteworthy that the value of input variable $\varphi(x,t)$ can be promptly determined for each sample point $(x,t)$.

#### 3.2.1 With Identified Discontinuity Locations

We now introduce our deep learning algorithm under the residual-minimization framework [57], starting with the generation of collocation points

$$X_{\text{Intrr}} = \left\{(x_n,t_n)\right\}_{n=1}^{N_{\text{Intrr}}}, \quad X_{\text{Shock}} = \left\{(x_n,t_n)\right\}_{n=1}^{N_{\text{Shock}}},$$
$$X_{\text{Bndry}} = \left\{(x_n,t_n)\right\}_{n=1}^{N_{\text{Bndry}}}, \quad X_{\text{Initl}} = \left\{(x_n,0)\right\}_{n=1}^{N_{\text{Initl}}},$$

that are sampled uniformly at random from the interior region $\Omega \times (0,T] \setminus \Gamma$, surfaces of discontinuity $\Gamma$, boundaries $\partial\Omega \times (0,T)$ and $\Omega \times \{t=0\}$ of the problem (1). Here, $N_{\text{Intrr}}$, $N_{\text{Shock}}$, $N_{\text{Bndry}}$, and $N_{\text{Initl}}$ denote the batch sizes of training datasets $X_{\text{Intrr}}$, $X_{\text{Shock}}$, $X_{\text{Bndry}}$, and $X_{\text{Initl}}$, respectively.

***Remark 2*** Critically, our solution ansatz is defined on piecewise surfaces within a one-order higher-dimensional space (see Figure 3a or Figure 4a), rather than fulfilling the entire $(x,t,\varphi)$-space. Hence, the number of collocation points remains unchanged regardless of the increased dimensionality, which is highly desirable for sample generation.

Next, by defining empirical loss functions according to the residual of equations (3)

$$
\begin{aligned}
L_{\text{Intrr}}(\theta) &= \frac{1}{N_{\text{Intrr}}} \sum_{n=1}^{N_{\text{Intrr}}} \left| \partial_t \hat{u}(x_n, t_n, \varphi(x_n, t_n); \theta) - \nabla_x \cdot f(\hat{u}(x_n, t_n, \varphi(x_n, t_n); \theta)) \right|^2, \\
L_{\text{Shock}}(\theta) &= \frac{1}{N_{\text{Shock}}} \sum_{n=1}^{N_{\text{Shock}}} \Big| s(x_n, t_n) [\![ \hat{u}(x_n, t_n, \varphi(x_n, t_n); \theta) ]\!] \\
&\qquad - \sum_{j=1}^{d} \nu_j [\![ f(\hat{u}(x_n, t_n, \varphi(x_n, t_n); \theta)) ]\!] \Big|^2, \\
L_{\text{Bndry}}(\theta) &= \frac{1}{N_{\text{Bndry}}} \sum_{n=1}^{N_{\text{Bndry}}} \left| \hat{u}(x_n, t_n, \varphi(x_n, t_n); \theta) - g(x_n, t_n) \right|^2, \\
L_{\text{Initl}}(\theta) &= \frac{1}{N_{\text{Initl}}} \sum_{n=1}^{N_{\text{Initl}}} \left| \hat{u}(x_n, 0, \varphi(x_n, 0); \theta) - u_0(x_n) \right|^2,
\end{aligned}
$$

the minimization task associated with our deep learning algorithm takes on the form

$$
\theta^* = \arg\min_{\theta} \; L_{\text{Intrr}}(\theta) + \beta_{\text{S}} L_{\text{Shock}}(\theta) + \beta_{\text{B}} L_{\text{Bndry}}(\theta) + \beta_{\text{I}} L_{\text{Initl}}(\theta), \tag{12}
$$

where $\beta_{\text{S}}$, $\beta_{\text{B}}$, and $\beta_{\text{I}} > 0$ are user-defined penalty parameters. Readers are directed to Algorithm 1 for a detailed illustration of the corresponding lift-and-embed learning algorithm.

**Algorithm 1** Lift-and-Embed Learning Algorithm with Identified Discontinuity Locations

*% Preparation (hyperparameter configurations: see Table 1 and our* ***Data Availability*** *statement)*

– construct the augmented variable with identified discontinuity locations

$$
\varphi(x, t) = H(x_d - \gamma(t, x_1, \cdots, x_{d-1}))
$$

where $\Gamma = \{(t, x_1, \cdots, x_d) \,|\, x_d = \gamma(t, x_1, \cdots, x_{d-1})\}$;

– generate training datasets $X_{\text{Intrr}}$, $X_{\text{Shock}}$, $X_{\text{Bndry}}$, and $X_{\text{Initl}}$, then compute their augmented variables;

*% Training Process*

– construct and initialize the neural network model

$$
\hat{u}(x, t, \varphi; \theta) = (L_{D+1} \circ \sigma \circ L_D \circ \cdots \circ \sigma \circ L_1 \circ L_0)(x, t, \varphi);
$$

**while** maximum number of epochs is not reached **do**

– draw mini-batch data uniformly at random from training datasets;

– network training on the shuffled dataset with a suitable learning rate, i.e.,

$$
\theta^* = \arg\min_{\theta} \; L_{\text{Intrr}}(\theta) + \beta_{\text{S}} L_{\text{Shock}}(\theta) + \beta_{\text{B}} L_{\text{Bndry}}(\theta) + \beta_{\text{I}} L_{\text{Initl}}(\theta);
$$

**end while**

*% Testing Process*

– forward pass of the trained model on the testing dataset, i.e.,

$$
\breve{u}(x, t) = \hat{u}(x, t, \varphi(x, t); \theta^*).
$$

### 3.2.2 With Unknown Discontinuity Locations

Differing from the previous study, we now explore scenarios in which the location of discontinuity is not known *a-priori*. Here, we will focus on the one-dimensional inviscid Burgers' equations, as the discontinuity curves of linear equations could typically be ascertained due to the parallelism of characteristic lines.

Note that in the one-dimensional case, the shock curve has a parametrization of the form $\Gamma = (t, \gamma(t))$ where $\gamma(0) = x_0$ and $s(t) = \frac{d\gamma(t)}{dt}$ [28]. Then, given an approximated shock speed $\hat{s}^{[k]}(t)$ at the $k$-th training epoch, the corresponding shock curve $\hat{\gamma}^{[k]}(t)$ can be determined by solving ordinary differential equations

$$\begin{cases} \dfrac{d\hat{\gamma}^{[k]}(t)}{dt} = \hat{s}^{[k]}(t), \quad \text{for } t \in (0, T], \\ \hat{\gamma}^{[k]}(0) = x_0. \end{cases} \tag{13}$$

Then, the augmented variable reads

$$\varphi^{[k]}(x, t) = H(x - \hat{\gamma}^{[k]}(t)),$$

followed by executing Algorithm 1 to obtain the trained solution $\hat{u}^{[k]}(x, t, \varphi^{[k]}(x, t); \theta^*)$. This, in turn, facilitates the employment of Rankine-Hugoniot jump condition (2) to adjust the value of shock speed for the next iteration, namely,

$$\hat{s}^{[k+1]}(t) = \arg\min_{\hat{s}(t)} \int_{\Gamma} \left| \hat{s}(t) [\![ \hat{u}^{[k]}(x, t, \varphi^{[k]}(x, t); \theta^*) ]\!] - [\![ f(\hat{u}^{[k]}(x, t, \varphi^{[k]}(x, t); \theta^*)) ]\!] \right|^2 dS.$$

In other words, both neural network model and shock speed are evaluated within the same framework, in which physics-informed machine learning [37, 57] has proven its flexibility and effectiveness.

To numerically realize the inference of shock curve alongside the training of neural network, the problem (13) is solved with instantaneous values $\{\hat{s}_i^{[k]} = \hat{s}^{[k]}(t_i)\}_{i=1}^n$ on discretized grids $0 = t_1 < \cdots < t_n = T$ to get $\{\hat{\gamma}^{[k]}(t_i)\}_{i=1}^n$. Then, the numerical shock speed $\hat{s}^{[k]}(t)$ and shock curve $\hat{\gamma}^{[k]}(t)$ can be determined by constructing their interpolating polynomials from datasets $\{\hat{s}^{[k]}(t_i)\}_{i=1}^n$ and $\{\hat{\gamma}^{[k]}(t_i)\}_{i=1}^n$ [62], respectively. Furthermore, the parametrized shock speed $\{\hat{s}_i^{[k+1]} = \hat{s}^{[k+1]}(t_i)\}_{i=1}^n$ can be integrated as leaf nodes within the existing computational graph, enabling simultaneous update with our neural network solution through a single backpropagation process.

More specifically, by including $\{\hat{s}_i\}_{i=1}^n$ as trainable parameters and defining the loss function (the superscript $[k]$ is omitted for notational simplicity)

$$\begin{aligned} L_{\text{Shock}}^{\text{inv}}(\theta, \{\hat{s}_i\}_{i=1}^n) = L_{\text{Shock}}(\theta) + \frac{1}{n} \sum_{i=1}^{n} \Big| & \hat{s}_i [\![ \hat{u}(\gamma(t_i), t_i, \varphi(\gamma(t_i), t_i); \theta) ]\!] \\ & - [\![ f(\hat{u}(\gamma(t_i), t_i, \varphi(\gamma(t_i), t_i); \theta)) ]\!] \Big|^2, \end{aligned} \tag{14}$$

the shock speed is co-updated with our solution ansatz through the backpropagation of neural network, as illustrated in Algorithm 2. Notably, the loss function (14) preserves a similar formulation as that in (12), despite the inclusion of extra collocation points for numerical integration.

**Algorithm 2** Lift-and-Embed Learning Algorithm with Unknown Discontinuity Locations

*% Preparation (hyperparameter configurations: see Table 1 and our* ***Data Availability*** *statement)*

– generate training datasets $X_{\text{Intr}}$, $X_{\text{Bndry}}$, $X_{\text{Initl}}$, and the discretization grids $\{t_i\}_{i=1}^n$;

– initialize $\{\hat{s}_i\}_{i=1}^n$ and add them into the optimizer as extra trainable parameters;

*% Training Process*

– construct and initialize the neural network model

$$\hat{u}(x,t,\varphi;\theta) = (L_{D+1}\circ\sigma\circ L_D\circ\cdots\circ\sigma\circ L_1\circ L_0)(x,t,\varphi);$$

**while** maximum number of epochs is not reached **do**

*% Identification of Shock Speed and Shock Curve*

– numerically solve (13) using discrete data $\{\hat{s}(t_i)=\hat{s}_i\}_{i=1}^n$, e.g., explicit Runge-Kutta schemes,

$$\{\hat{\gamma}(t_i)\}_{i=1}^n = \text{ODE_Solver}(\{\hat{s}(t_i)\}_{i=1}^n, \hat{\gamma}(0)=x_0, \{t_i\}_{i=1}^n);$$

– reconstruct $\hat{s}(t)$ and $\hat{\gamma}(t)$ from $\{\hat{s}(t_i)\}_{i=1}^n$ and $\{\hat{\gamma}(t_i)\}_{i=1}^n$, e.g. interpolations;

– update the augmented variable $\varphi(x,t)$ and generate the training dataset $X_{\text{Shock}}$

$$\varphi(x,t) = H(x-\hat{\gamma}(t)) \text{ where } \Gamma=\{(x,t)\,|\,x=\hat{\gamma}(t)\};$$

*% Update of Network Solution and Shock Speed Concurrently*

– draw mini-batch data uniformly at random from training datasets;

– network training on the shuffled dataset with a suitable learning rate, i.e.,

$$\theta^*, \{\hat{s}_i^*\}_{i=1}^n = \arg\min_{\theta,\ \hat{s}_i} L_{\text{Intr}}(\theta) + \beta_{\text{S}} L_{\text{Shock}}^{\text{inv}}(\theta, \{\hat{s}_i\}_{i=1}^n) + \beta_{\text{B}} L_{\text{Bndry}}(\theta) + \beta_{\text{I}} L_{\text{Initl}}(\theta);$$

**end while**

*% Testing Process*

– forward pass of the trained model on the testing dataset, i.e.,

$$\breve{u}(x,t) = \hat{u}(x,t,\varphi(x,t);\theta^*).$$

## 4 Numerical Experiments

In this section, we conduct numerical studies on a series of benchmark problems to demonstrate the effectiveness and flexibility of our methods. More precisely, the inclusion of an augmented variable allows for a representation of jump conditions as differences between distinct collocation points, while the parametrization of solution ansatz with neural networks can mitigate the curse of dimensionality and handle shock waves that may propagate along lines, curves, or a combination of both. Our approach is also capable of addressing both linear and nonlinear equations, including cases with unknown discontinuity interfaces, within a unified learning framework. We also extend our study to the hyperbolic system, particularly the Euler equations, using the Sod shock tube problem as a standard benchmark test. Upon the completion of the training process, the approximate solution can be obtained by projecting the trained neural network solution back onto the original physical domain.

**Table 1** List of key hyper-parameter configurations for numerical experiments

| | | Neural Network (Depth, Width) | Penalty Coeff. ($\beta_S$, $\beta_B$, $\beta_I$) | Train Data Size ($N_{Intrr}$, $N_{Shock}$, $N_{Bndry}$, $N_{Initl}$) | Test Data Size ($N_x$, $N_t$) |
|---|---|---|---|---|---|
| Linear Convection Equations | Sec. 3.1.1 | (2, 40) | (400, 1, 400) | ($10k$, $4k$, $2k$, $1k$) | ($1k$, $1k$) |
| | Sec. 4.1.1 | (2, 40) | (400, 1, 400) | ($10k$, $4k$, $2k$, $1k$) | ($1k$, $1k$) |
| | Sec. 4.1.2 | (6, 40) | (400, 10, 400) | ($80k$, $60k$, $5k$, $5k$) | ($1k$, $1k$) |
| Inviscid Burgers Equations | Sec. 3.1.2 | (3, 40) | (400, 1, 400) | ($10k$, $1k$, $1k$, $1k$) | ($1k$, $1k$) |
| | Sec. 4.2.1 | (6, 40) | (400, 1, 400) | ($80k$, $15k$, $5k$, $5k$) | ($1k$, $1k$) |
| | Sec. 4.2.2 | (6, 40) | (400, 1, 400) | ($80k$, $10k$, $10k$, $5k$) | ($1k$, $1k$) |
| | Sec. 4.2.3 | (4, 80) | (50, 1, 400) | ($80k$, $20k$, $30k$, $10k$) | ($1k \times 1k$, 17) |
| | Sec. 4.3.1 | (4, 40) | (400, 1, 400) | ($10k$, $1k$, $1k$, $1k$) | ($1k$, $1k$) |
| | Sec. 4.3.2 | (6, 40) | (50, 1, 400) | ($80k$, $5k$, $5k$, $5k$) | ($1k$, $1k$) |
| Buckley–Leverett Equation | Sec. 4.4 | (6, 40) | (10, 1, 400) | ($40k$, $1k$, $1k$, $1k$) | ($1k$, $1k$) |
| Euler Equations | Sec. 4.5 | (6, 40) | (20, 0, 40) | ($80k$, $5k$, 0, $5k$) | ($1k$, $1k$) |

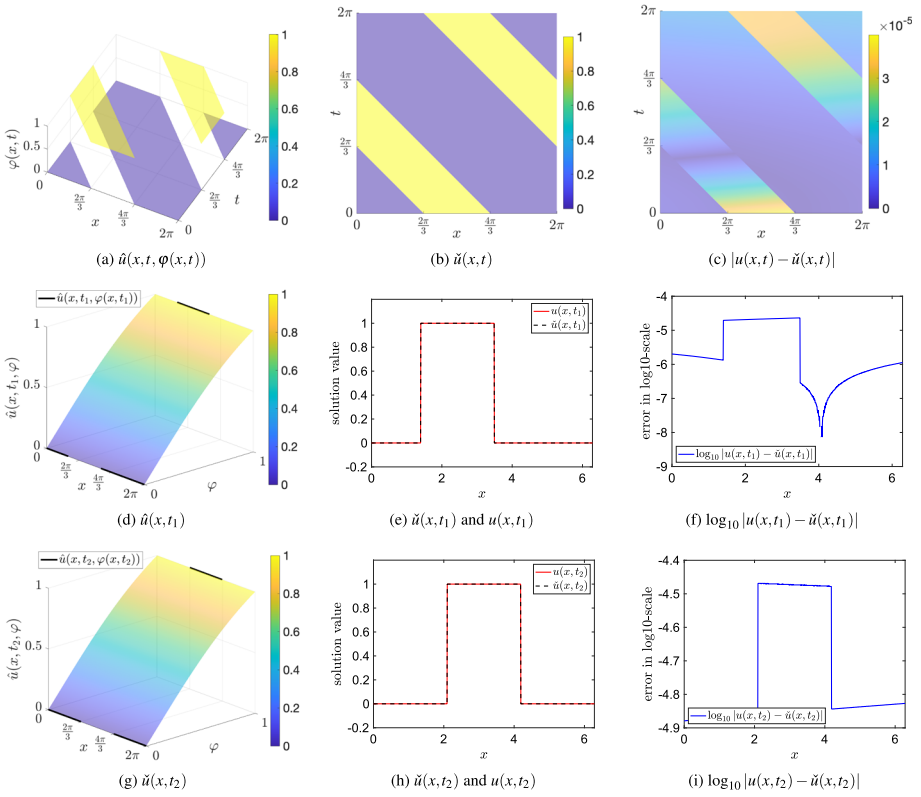



**Fig. 5** Numerical results for the convection equation (4) using another augmented variable ($t_1 = \frac{2\pi}{9}, t_2 = 2\pi$)

Note that our method offers the merit of using a smooth neural network to learn discontinuous solutions, hence the fully-connected neural network is deployed with the use of hyperbolic tangent activation functions [29]. For the following numerical experiments, different hyper-parameter configurations are implemented and summarized in Table 1. When the maximum number of training epochs is reached, the model with the smallest training loss, indicated by $\hat{u}(x,t,\varphi(x,t))$ as before, is adopted for comparison with the exact solution $u(x,t)$, while the projected solution onto the original $(x,t)$-plane is indicated as $\check{u}(x,t)$. All network models are trained using the ADAMW optimizer [39, 49], with an initial learning rate set to 0.01 and a step decay schedule that divides it by 10 after specific milestones.

To assess the performance of our trained neural network solutions, we generate testing points $\{(x_i,t_j)\}_{i=1,\,j=1}^{N_x,\,N_t}$ through a uniform partition of the computational domain, and then report the relative $L^2$ error in Table 2, i.e.,

$$\frac{\|u(x,t)-\check{u}(x,t)\|_2}{\|u(x,t)\|_2} \quad \text{where} \quad \|u\|_2 = \sqrt{\sum\nolimits_{m=1}^{M} |u(x_m,t_m)|^2}.$$

All experiments are conducted using PyTorch on Nvidia GeForce RTX 4090 GPU cards [54].

**Table 2** Relative $L^2$ errors for numerical experiments conducted in this study

| | | | | | | |
|---|---|---|---|---|---|---|
| Linear Convection Equations | Sec. 3.1.1 | Sec. 4.1.1 | Sec. 4.1.2 | | | |
| | $2.01 \times 10^{-3}$ | $2.49 \times 10^{-5}$ | $7.03 \times 10^{-4}$ | | | |
| Inviscid Burgers Equations | Sec. 3.1.2 | Sec. 4.2.1 | Sec. 4.2.2 | Sec. 4.2.3 | Sec. 4.3.1 | Sec. 4.3.2 |
| | $9.47 \times 10^{-5}$ | $1.85 \times 10^{-4}$ | $4.85 \times 10^{-3}$ | $1.38 \times 10^{-5}$ | $1.82 \times 10^{-2}$ | $3.64 \times 10^{-2}$ |
| Buckley–Leverett Equation | Sec. 4.4 | | | | | |
| | $1.73 \times 10^{-3}$ | | | | | |
| Euler Equations ($\rho$, $u$, $p$) | Sec. 4.5 | Sec. 4.5 | Sec. 4.5 | | | |
| | $7.66 \times 10^{-3}$ | $5.20 \times 10^{-2}$ | $8.06 \times 10^{-3}$ | | | |

### 4.1 Linear Convection Equations

In this section, we report numerical studies on linear convection equations with a discontinuous initial condition, as well as a large convection coefficient [17, 41], to validate the effectiveness of our methods. Algorithm 1 is used to train the neural network solution.

#### 4.1.1 The Benchmark Linear Convection Problem Revisited

By revisiting the benchmark linear convection problem (4), we showcase another way of building the augmented variable, as the additional degree of freedom offers non-unique mappings that yield the identical behaviour upon collapsing to the original lower-dimensional domain. To be precise, we consider

$$u(x,t) = \hat{u}(x,t,\varphi(x,t)) \quad \text{with} \quad \varphi(x,t) = \sum_{k=0}^{n} u_0(x - st - 2k\pi),$$

where $s = -1$, $n = \left\lceil -\frac{sT+x_1}{2\pi} \right\rceil$ with $x_1 = \frac{2\pi}{3}$ and $T = 2\pi$. Here, the augmented variable $\varphi(x,t)$ is constructed using the initial condition (5), which also enables the representation of jump discontinuities as differences across distinct collocation points (see Figure 5a). The lifted hyperbolic system adopts the same formulation as that of (8) and the corresponding computational results are reported in Figure 5 and Table 2.

As can be seen from (b), (e), and (h) in Figure 5, our trained neural network solution can effectively capture the discontinuous solution without notable smearing or oscillation, even as the time evolves to $t = 2\pi$. For comparison, readers are referred to textbook [30] for the numerical solution obtained using classical finite difference methods. Furthermore, superior accuracy is achieved compared to that in Figure 3, as the augmented variable, or equivalently, the input data of our neural network, coincides with the exact solution for this specific setting.

#### 4.1.2 Discontinuous Solution with A Large Convection Coefficient

Recent empirical studies suggest that standard learning methods may struggle to resolve the exact solution when the convection coefficient becomes large [15], therefore requiring complex adjustments like sequence-to-sequence learning [41] or retain-resample-release sampling [17]. To wrap up our study on linear problems, we explore a scenario marked by not only a large convection coefficient but also a discontinuous initial condition, that is,

$$\begin{cases} \partial_t u(x,t) - 50\partial_x u(x,t) = 0, & \text{for } (x,t) \in \Omega = (0,2\pi) \times (0, \frac{\pi}{5}], \\ u_0(x) = H(x - \frac{2\pi}{3}) - H(x - \frac{4\pi}{3}), & \text{for } x \in (0, 2\pi), \\ u(0,t) = u(2\pi, t), & \text{for } t \in (0, \frac{\pi}{5}]. \end{cases} \tag{15}$$

Furthermore, the augmented variable for (15) is formulated as a step function to avoid potential coincidence with the exact solution. To be precise, we employ the identical configuration for solution representation as outlined in (6), with the only distinction being the presence of a large convection coefficient within the current problem.

The neural network solution and its projection back to the lower-dimensional flat plane are shown in Figure 6, as well as error profiles over the entire domain or at specific temporal slices (see (c), (f), and (i) in Figure 6). Clearly, our numerical solution closely matches the exact solution without spurious smearing and oscillation (see Figure 6e and Figure 6h), regardless of the presence of numerous discontinuity interfaces incurred by the large convection coefficient.

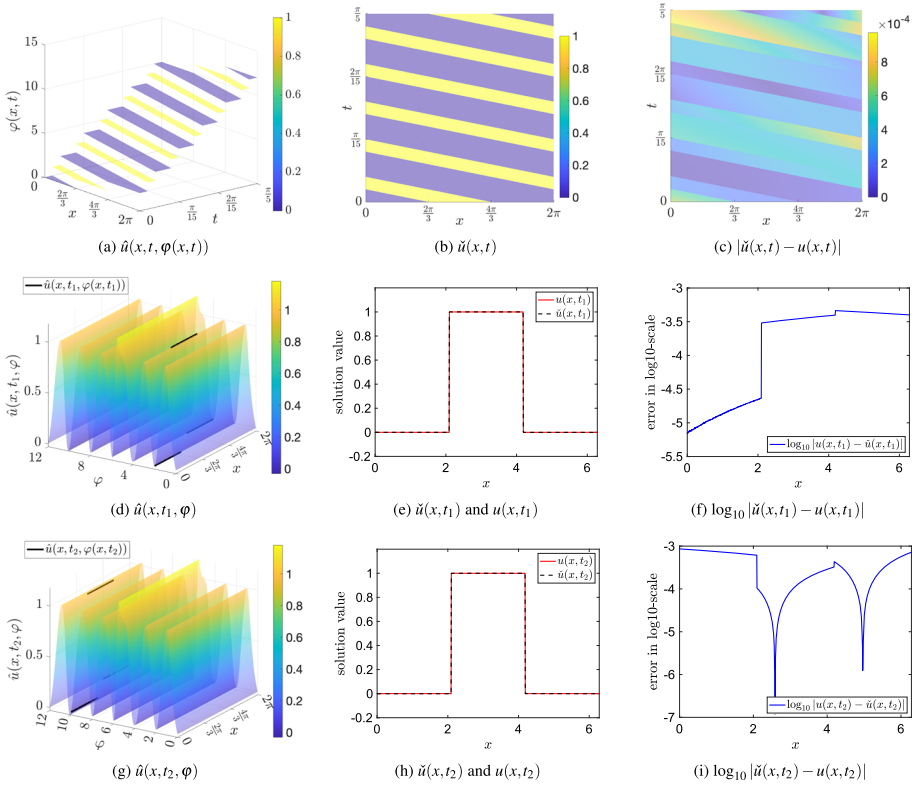



**Fig. 6** Numerical results for the convection equation (15) with a large convection coefficient ($t_1 = \frac{\pi}{25}, t_2 = \frac{\pi}{5}$)

## 4.2 Inviscid Burgers' Equations with Identified Shock Locations

Next, we present numerical studies on the one-dimensional inviscid Burgers' equation, focusing on shock-shock and rarefaction-shock interactions, to show the effectiveness of our methods. Following this, a two-dimensional problem is examined in Section 4.2.3 to showcase the neural network's capability in tackling high-dimensional functions. Here, Algorithm 1 is again used to train the neural network solution.

### 4.2.1 Shock-Shock Interaction

To begin with, we consider the inviscid Burgers' equation in the one dimension, i.e.,

$$\begin{cases} \partial_t u(x,t) + u(x,t)\partial_x u(x,t) = 0, & \text{for } (x,t) \in \Omega = (0,3) \times (0,2], \\ u_0(x) = \begin{cases} x, & \text{for } 0 \le x < 1, \\ 0, & \text{for } 1 \le x < 2, \\ -2, & \text{for } 2 \le x \le 3, \end{cases} \\ u(0,t) = 0,\ u(3,t) = -2, & \text{for } t \in (0,2], \end{cases} \tag{16}$$

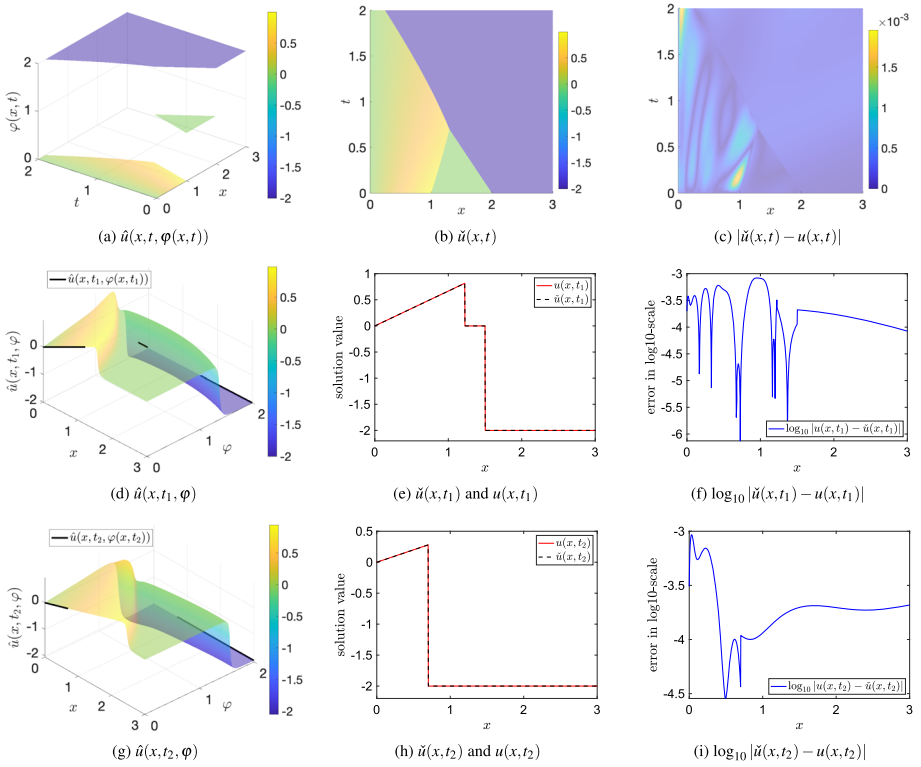



**Fig. 7** Numerical results for the Burgers' equation (16) with two shocks coalesce into one ($t_1 = 0.5, t_2 = 1$)

which can be decomposed into two Riemann problems at $x_0 = 1$ and $x_0 = 2$. As depicted in Figure 1c, the characteristic line emanating from $x_0 \in (0, 1)$ takes on the form

$$x(t) = u_0(x_0)t + x_0 = x_0(1+t) \quad \text{with} \quad u(x,t) = u_0(x_0) = \frac{x}{1+t},$$

while that departing from $x_0 \in (1, 2)$ is vertical with $u(x,t) = u_0(x_0) = 0$. Consequently, the shock curve $x = \gamma_1(t)$ emerging from $x_0 = 1$ must satisfies the Rankine-Hugoniot jump condition (see Lemma 1), i.e.,

$$s_1(t) = \frac{d\gamma_1(t)}{dt} = \frac{1}{2}\left(\frac{\gamma_1(t)}{1+t} + 0\right) \quad \text{with} \quad \gamma_1(0) = 1,$$

which implies that the first shock curve reads $\gamma_1(t) = \sqrt{1+t}$. On the other hand, the second shock curve $x = \gamma_2(t)$ emerging from $x_0 = 2$ is given by

$$s_2(t) = \frac{d\gamma_2(t)}{dt} = \frac{1}{2}(0-2) \quad \text{with} \quad \gamma_2(0) = 2, \quad \text{or equivalently,} \quad \gamma_2(t) = -t + 2,$$

which will reach the right-propagating shock $\gamma_1(t)$ at the point $(x^*, t^*) = (\frac{\sqrt{13}-1}{2}, \frac{5-\sqrt{13}}{2})$.

At the moment when two shocks collide, we obtain

$$s_3(t) = \frac{d\gamma_3(t)}{dt} = \frac{1}{2}\left(\frac{\gamma_3(t)}{1+t} - 2\right) \quad \text{with} \quad \gamma_3(t^*) = x^*,$$

or equivalently, $\gamma_3(t) = \sqrt{13(1+t)} - 2(1+t)$, which indicates that two shocks merge to become a single shock at later times. Now, we are prepared to embed all these jump conditions by incorporating an augmented variable

$$\varphi(x,t) = \begin{cases} H(x-\gamma_1(t)) + H(x-\gamma_2(t)), & \text{for } 0 \le t \le t^*, \\ 2H(x-\gamma_3(t)), & \text{for } t^* < t \le 2, \end{cases}$$

and the hyperbolic system satisfied by our solution ansatz $\hat{u}(x,t,\varphi(x,t))$ is then defined as

$$\begin{cases} \partial_t \hat{u}(x,t,\varphi(x,t)) + \hat{u}(x,t,\varphi(x,t))\partial_x \hat{u}(x,t,\varphi(x,t)) = 0, & \text{for } (x,t) \in \Omega \setminus \bigcup_{i=1}^{3} \Gamma_i, \\ \hat{u}(x,t,\varphi^+(x,t)) + \hat{u}(x,t,\varphi^-(x,t)) = 2s_i(t), & \text{for } (x,t) \in \Gamma_i \ (1 \le i \le 3), \\ \hat{u}(x,0,\varphi(x,0)) = u_0(x), & \text{for } x \in (0,3), \\ \hat{u}(0,t,\varphi(0,t)) = 0, \ \hat{u}(3,t,\varphi(3,t)) = -2, & \text{for } t \in (0,2]. \end{cases}$$

Numerical results obtained through our lift-and-embed learning approach are displayed in Figure 7, which are in good agreement with the exact solution. Notably, shock waves of (16) not only intersect with each other but also appear as curved interfaces, which often incur higher costs for traditional numerical schemes requiring interface-fitted meshes. Thanks to the meshless nature of neural networks, our method can effectively adapt to irregular shock curves (see Figure 7b and Figure 7c), making it a compelling choice for scenarios with complex geometries of shock trajectories.

Additionally, as depicted in Figure 7d and Figure 7g, temporal slices of trained network solution are now situated on surfaces within a one-order higher-dimensional space. All solution jumps of (16) are reconstructed as differences between distinct collocation points (see Figure 7d and Figure 7g), which are effectively reconstructed using a fully-connected neural network with smooth activation functions.

### 4.2.2 Rarefaction-Shock Interaction

In this numerical experiment, we consider another inviscid Burgers' equation, i.e.,

$$\begin{cases} \partial_t u(x,t) + \frac{1}{2} u(x,t)\partial_x u(x,t) = 0, & \text{for } (x,t) \in \Omega = (-1,6) \times (0,10], \\ u_0(x) = \begin{cases} 0, & \text{for } -1 \le x \le 0, \\ 1, & \text{for } 0 < x < 1, \\ 0, & \text{for } 1 \le x \le 6, \end{cases} \\ u(-1,t) = u(6,t) = 0, & \text{for } t \in (0,10], \end{cases} \tag{17}$$

which is decomposed into two Riemann problems at $x_0 = 0$ and $x_0 = 1$. However, characteristics do not cover the whole plane as shown in Figure 1d, leading to the formation of a rarefaction wave between $x = 0$ and $x = \frac{1}{2}t$. On the other hand, the shock wave originating form $x_0 = 1$ propagates along a straight line with the speed

$$s_1 = \frac{[\![f(u)]\!]}{[\![u]\!]} = \frac{1}{4}\left(u^+(x,t) + u^-(x,t)\right) = \frac{1}{4} \quad \text{where} \quad f(u) = \frac{1}{4}u^2.$$

As time progresses, the shock wave encounters the rarefaction wave at $(x,t) = (2,4)$, giving rise to another shock wave with a non-constant velocity. More precisely, this shock wave satisfies the Rankine-Hugoniot condition

$$s_2(t) = \frac{d\gamma_2(t)}{dt} = \frac{1}{4}\left(\frac{2\gamma_2(t)}{t} + 0\right) \quad \text{with} \quad \gamma_2(4) = 2,$$

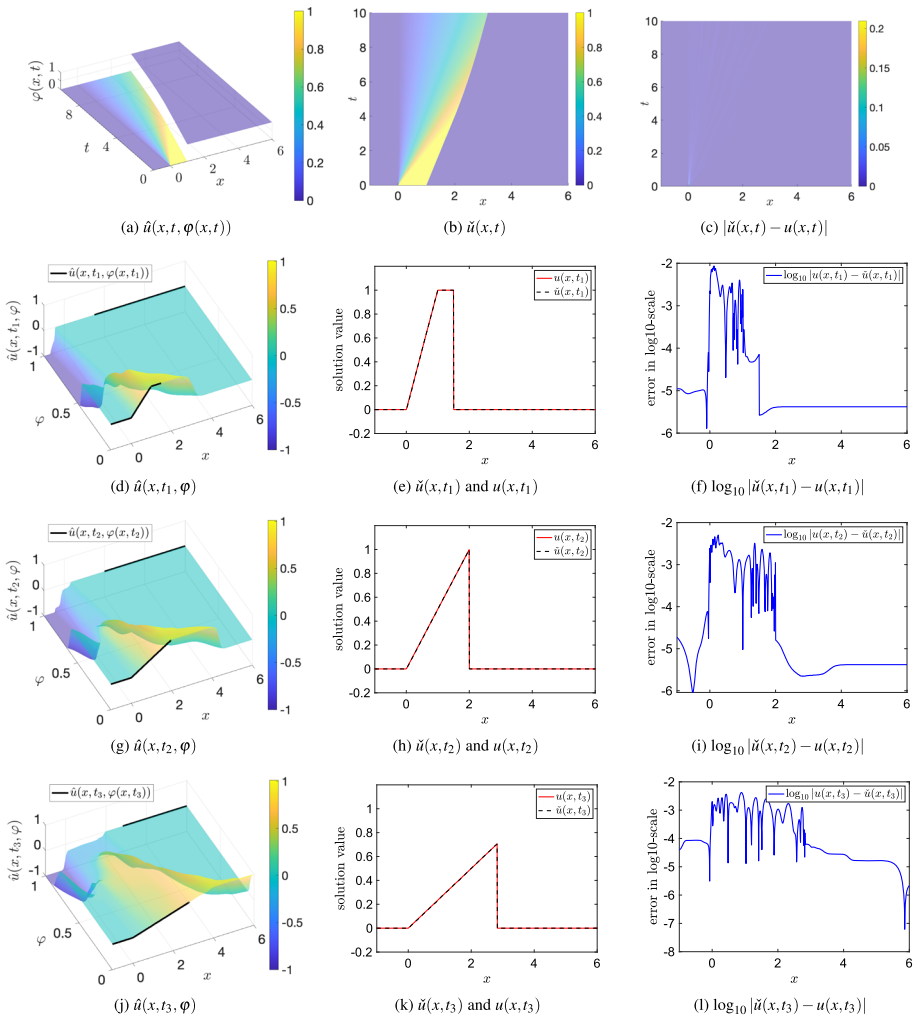



**Fig. 8** Numerical results for the Burgers' equation (16) with rarefaction-shock interaction ($t_1 = 2$, $t_2 = 4$, $t_3 = 8$)

which implies that $\gamma_2(t) = \sqrt{t}$ for $t > 4$. Consequently, our augmented variable can be defined as

$$\varphi(x,t) = H(x-\gamma(t)) \text{ with } \gamma(t) = \begin{cases} \frac{1}{4}t+1, & \text{for } t \in [0,4], \\ \sqrt{t}, & \text{for } t \in (4,10], \end{cases}$$

while the hyperbolic system satisfied by our solution ansatz $\hat{u}(x,t,\varphi(x,t))$ takes on the form

$$\begin{cases} \partial_t \hat{u}(x,t,\varphi(x,t)) + \frac{1}{2}\hat{u}(x,t,\varphi(x,t))\partial_x \hat{u}(x,t,\varphi(x,t)) = 0, & \text{for } (x,t) \in \Omega \setminus \bigcup_{i=1}^{2} \Gamma_i, \\ \hat{u}(x,t,\varphi^+(x,t)) + \hat{u}(x,t,\varphi^-(x,t)) = 4s_i(t), & \text{for } (x,t) \in \Gamma_i \ (1 \le i \le 2), \\ \hat{u}(x,0,\varphi(x,0)) = u_0(x), & \text{for } x \in (-1,6), \\ \hat{u}(-1,t,\varphi(-1,t)) = \hat{u}(6,t,\varphi(6,t)) = 0, & \text{for } t \in (0,10]. \end{cases}$$

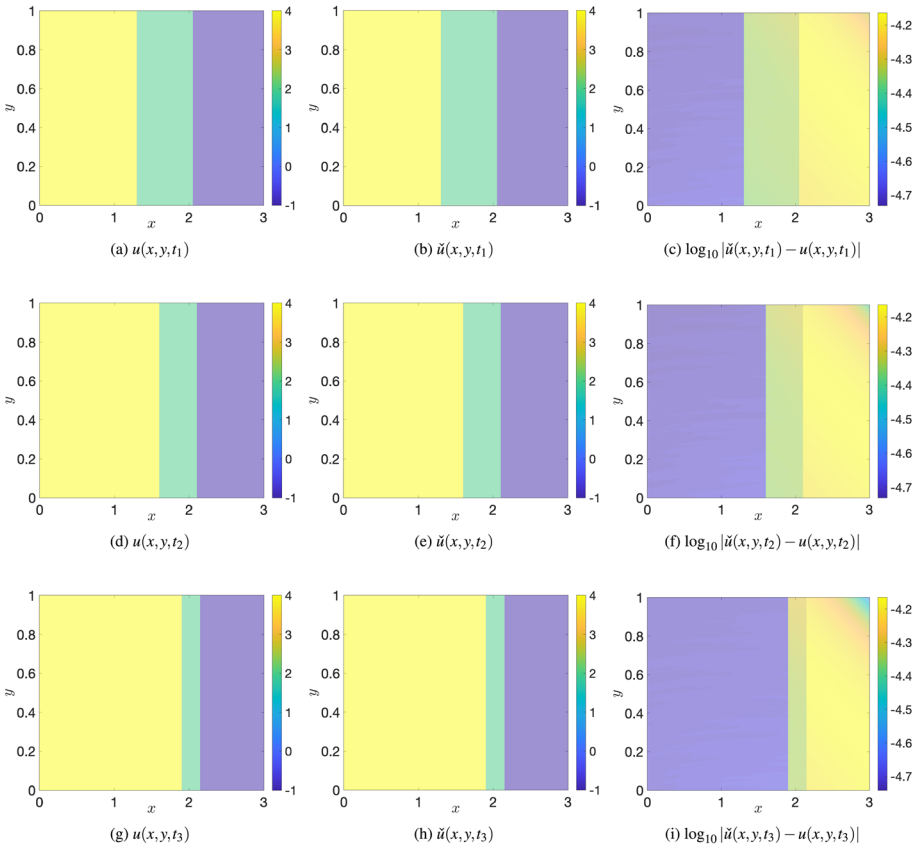


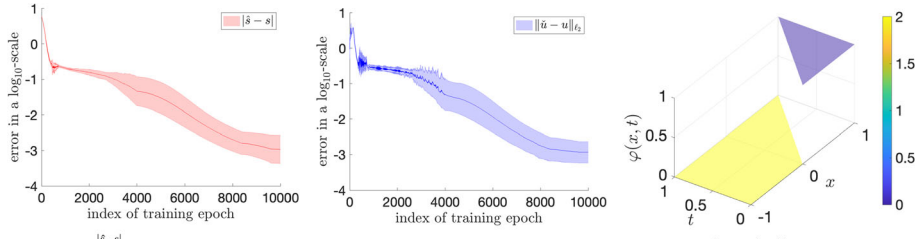

**Fig. 9** Numerical results for the inviscid Burgers' equation (18) in two-dimension ($t_1 = 0.1$, $t_2 = 0.2$, $t_3 = 0.3$)

Numerical results obtained through our lift-and-embed learning approach are shown in Fiigure 8, demonstrating a satisfactory approximation of both rarefaction and shock waves. During the training process, an additional penalty coefficient $\beta = 300$ is assigned to the interior loss term (12) in this case. Notably, the approximation error associated with the rarefaction wave is comparatively higher than that observed in other regions (see Fiigure 8f, Fiigure 8i, and Fiigure 8l for instance), which is likely to be further improved by incorporating entropy conditions into the training loss function [18] (a subject left for future investigation).

### 4.2.3 Two-Dimensional Inviscid Burgers' Equation

Here, we study a problem on the domain
$\Omega = (0, 3) \times (0, 1) \times (0, 0.4]$ to show the capacity in tackling high-dimensional functions,

$$\begin{cases} \partial_t u(x,y,t) + u(x,y,t)\partial_x u(x,y,t) + u(x,y,t)\partial_y u(x,y,t) = 0, & \text{for } (x,y,t) \in \Omega = (0,3)\times(0,1)\times(0,0.4], \\ u_0(x) = \begin{cases} 4, & \text{for } 0 \le x < 1, \\ 2, & \text{for } 1 \le x < 2, \\ -1, & \text{for } 2 \le x \le 3, \end{cases} \end{cases} \tag{18}$$




(a) $\log_{10}^{|\hat{s}-s|}$ (mean ± deviation) (b) test loss (mean ± deviation) (c) $\hat{u}(x,t,\varphi(x,t))$ in one run

**Fig. 10** Numerical results for the Burgers' equation (3.1.2) with the initial guess of shock speed set to $\hat{s} = -5$

with inflow boundary conditions derived from the exact solution

$$u(x, y, t) = 4H(1 + 3t - x) + 2H(x - 1 - 3t) - 3H\left(x - \frac{1}{2}t - 2\right).$$

We denote by $\Gamma_i = \Gamma_i(x, y, t)$ the surface of discontinuities departing from the initial position $\{(x_i, y, 0) \,|\, 0 \le y \le 1\}$ and $\boldsymbol{n}_i = (-s_i, \boldsymbol{v}_i)^T$ the normal vector to it, in which $1 \le i \le 2$, $x_1 = 1$, and $x_2 = 2$. Due to the constancy of initial values, both surfaces are planar where $\boldsymbol{n}_1 = (-3, 0, 1)^T$ and $\boldsymbol{n}_2 = (-0.5, 0, 1)^T$, and hence can be parametrized as

$$x = 1 + 3t \quad \text{and} \quad x = 2 + 0.5t$$

for $0 \le y \le 1$, respectively. Accordingly, the augmented variable can be constructed as

$$\varphi(x, y, t) = H(x - 2 - 0.5t) + H(x - 1 - 3t),$$

leading to a four-dimensional function $\hat{u}(x, y, t, \varphi(x, y, t))$ to be resolved from the embedded version of (18), i.e.,

$$\begin{cases} \partial_t \hat{u}(x, y, t, \varphi(x, y, t)) + 0.5[\partial_x + \partial_y]\hat{u}^2(x, y, t, \varphi(x, y, t)) = 0, & \text{for } (x, y, t) \in \Omega \setminus \bigcup_{i=1}^{2} \Gamma_i, \\ -s_i[\![\hat{u}(x, y, t, \varphi(x, y, t))]\!] + 0.5[\![\hat{u}^2(x, y, t, \varphi(x, y, t))]\!] = 0, & \text{for } (x, y, t) \in \Gamma_i \ (i = 1, 2), \\ \hat{u}(x, 0, \varphi(x, 0)) = u_0(x), & \text{for } x \in (0, 3), \end{cases}$$

with inflow boundary conditions. Traditional mesh-based numerical methods are often susceptible to the curse of dimensionality, whereas neural network-based approaches offer a promising way to alleviate this issue.

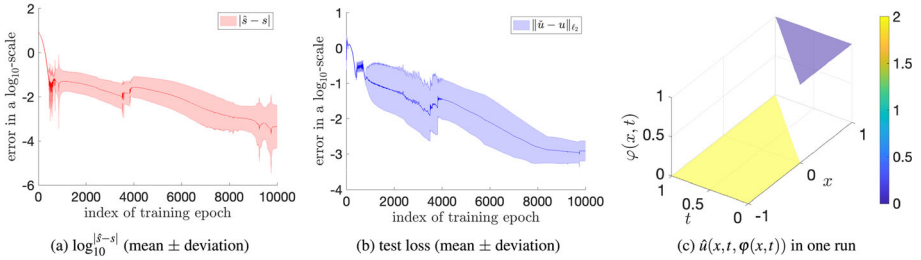
As illustrated in Figure 9 and Table 2, our approximate solution shows reliable alignment with the true solution (only specific temporal slices are depicted due to the high dimensionality), which demonstrates the capability of our proposed method in addressing high-dimensional discontinuous solutions.

### 4.3 Inviscid Burgers' Equations with Unknown Shock Locations

Unlike previous examples, this section addresses the inviscid Burgers' equation without specifying the location of discontinuities in advance. Instead, Algorithm 2 is utilized to dynamically infer shock curves during training.

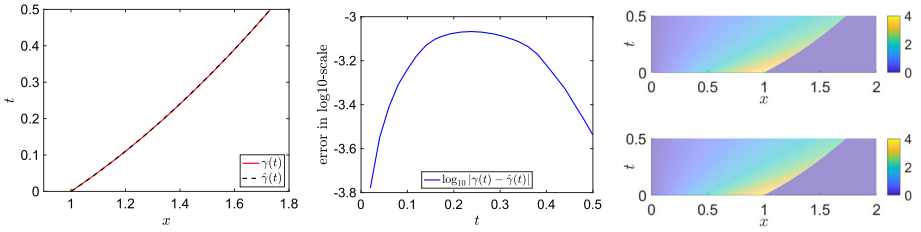
#### 4.3.1 The Benchmark Inviscid Burgers' Problem Revisited

Here, we re-examine the inviscid Burgers' equation (9) in which the shock curve takes the form of a straight line $\gamma(t) = st$. In contrast to the previous study, the shock speed $s \in \mathbb{R}$,




(a) $\log_{10}^{|\hat{s}-s|}$ (mean $\pm$ deviation) (b) test loss (mean $\pm$ deviation) (c) $\hat{u}(x,t,\varphi(x,t))$ in one run

**Fig. 11** Numerical results for the Burgers' equation (9) with the initial guess of shock speed set to $\hat{s} = 10$




(a) $\gamma(t)$ and $\hat{\gamma}(t)$ (b) $\log_{10}|\gamma(t)-\hat{\gamma}(t)|$ (c) $u(x,t)$ and $\check{u}(x,t)$

**Fig. 12** Numerical results for the Burgers' equation (19) with a curved shock trajectory $\gamma(t) = \sqrt{1+4t}$

as integrated into the formulation of our ansatz (10), is regarded as an unknown parameter to be inferred. Specifically, the empirical loss function (14) now reads

$$L^{\text{inv}}_{\text{Shock}}(\theta, \hat{s}) = L_{\text{Shock}}(\theta) + \frac{1}{n}\sum_{i=1}^{n} \left| \hat{s}[\![\hat{u}(\gamma(t_i), t_i, \varphi(\gamma(t_i), t_i); \theta)]\!] - [\![f(\hat{u}(\gamma(t_i), t_i, \varphi(\gamma(t_i), t_i); \theta))]\!] \right|^2.$$

The initial guess of shock speed $\hat{s}$ is set to $-5$ and 10 respectively, while the true value $s = 1$ as discussed in Subsection 3.1.2. Algorithm 2 is evaluated over 6 independent runs, with the mean and standard deviation of parameter error $|\hat{s} - s|$ and testing loss $\|\check{u} - u\|_{\ell_2}$ being reported in Figure 10 and Figure 11, as well as our trained neural network solution $\hat{u}(x, t, \varphi(x, t))$ in a typical simulation. Predicted shock speeds are in good agreement with the exact value in both cases, which validates the flexibility of our learning approach. It is also noteworthy that the approximation accuracy in this case is slightly lower compared to that in Subsection 3.1.2 (see Table 2), as the discrepancy between $\hat{s}$ and $s$ would compromise the training accuracy of our neural network solution.

### 4.3.2 Curved Shock Trajectory

Next, we consider the following inviscid Burgers' equation

$$\begin{cases} \partial_t u(x,t) + u(x,t)\partial_x u(x,t) = 0, & \text{for } (x,t) \in \Omega = (0,2) \times (0,0.5], \\ u_0(x) = \begin{cases} 4x, & \text{for } 0 \le x < 1, \\ 0, & \text{for } 1 \le x \le 2, \end{cases} \\ u(0,t) = u(2,t) = 0, & \text{for } t \in (0,0.5], \end{cases} \tag{19}$$

where the shock wave, departing from the initial discontinuity at $\gamma(0) = x_0 = 1$, propagates along a curved trajectory $\gamma(t) = \sqrt{1+4t}$. Note that, in contrast to the numerical exam-

ple reported in Subsection 4.3.1, the shock speed $s(t)$ in (19) varies over time and is thus discretized on an equidistant mesh with meshwidth $h = 1/50$.

We choose $\hat{\gamma}(t) = 0.5t + \bar{\gamma}(t)$ as our approximated shock curve, with a heuristic shock trajectory $\gamma(t) \approx 0.5t$ being used as the hypothesis model, which is determined by numerically solving (13), i.e.,

$$\begin{cases} \dfrac{d\bar{\gamma}(t)}{dt} = \hat{s}(t) - 0.5, & \text{for } t \in (0, 0.5], \\ \bar{\gamma}(0) = 1, \end{cases} \tag{20}$$

through a fourth-order Runge-Kutta method on the equidistant mesh $\{t_i = (i-1)h\}_{i=1}^{26}$ with the initial guess of instantaneous values set to $\{\hat{s}_i = \hat{s}(t_i) = 0\}_{i=1}^{26}$. Accordingly, the augmented variable can be constructed as

$$\varphi(x, t) = H(x - \hat{\gamma}(t)),$$

and Algorithm 2 is employed to train the neural network solution and infer the shock speed. Here, an additional penalty coefficient $\beta = 100$ is assigned to the interior loss term.

Numerical results obtained through our proposed learning approach are displayed in Figure 12, in which the shock curve and the discontinuous solution are recovered with commendable accuracy. Though the error arising from the numerical reconstruction of shock curve would further affect the accuracy of the trained neural network solution, the relative error reported in Table 2 maintains reasonably good performance.

### 4.4 Buckley-Leverett Equation with Unknown Shock Locations

Unlike the convex flux function studied before, we now consider the Buckley-Leverett equation [6, 43], namely,

$$\begin{cases} \partial_t u(x,t) + \partial_x \left( \dfrac{u^2(x,t)}{u^2(x,t) + \alpha(1-u(x,t))^2} \right) = 0, & \text{for } (x,t) \in \Omega = [-1,1] \times (0,1], \\ u_0(x) = \begin{cases} 1, \\ 0, \end{cases} & \begin{array}{l} \text{for } -1 \le x < 0, \\ \text{for } 0 \le x \le 1, \end{array} \\ u(0,t) = 1, & \text{for } t \in (0,1], \end{cases} \tag{21}$$

where $u(x,t) : \Omega \to [0,1]$ indicates the water saturation in an oil-water two-phase flow and $\alpha = 0.5$ the viscosity ratio. Owing to the non-convexity of the flux function [6], the discontinuous initial data evolves into a compound wave composed of a right-propagating shock with a rarefaction attached to it [43]. To be precise, the shock wave propagates at the same speed as the trailing edge of the rarefaction wave, requiring the Rankine-Hugoniot jump condition to be fulfilled at an entropy-compatible state $u_*(x,t) \in (0,1)$ rather than its original form (2), that is,

$$f'(u_*(x,t)) = s = \frac{f(u_*(x,t))}{u_*(x,t)} \quad \text{on } \Gamma = \{(x(t), t) \mid x(t) = st\},$$

where $s = \sqrt{\frac{\alpha}{\alpha+1}} = \frac{\sqrt{3}}{3}$ is the exact shock speed, $f(u) = \frac{u^2}{u^2+\alpha(1-u)^2}$, and $f'(u) = \frac{2\alpha u(1-u)}{(u^2+\alpha(1-u)^2)^2}$.

Within our learning framework, the shock speed is regarded as a trainable parameter $\hat{s} \in \mathbb{R}$ to be inferred via Algorithm 2, and the augmented variable is constructed as

$$\varphi(x,t) = H(x - \hat{s}t),$$

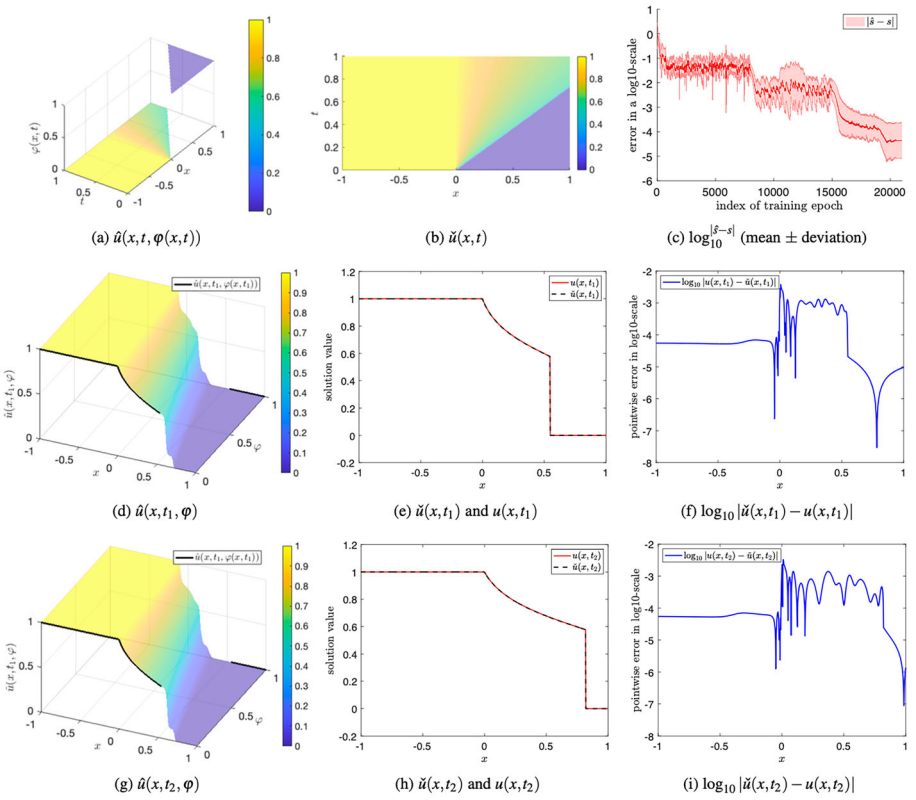


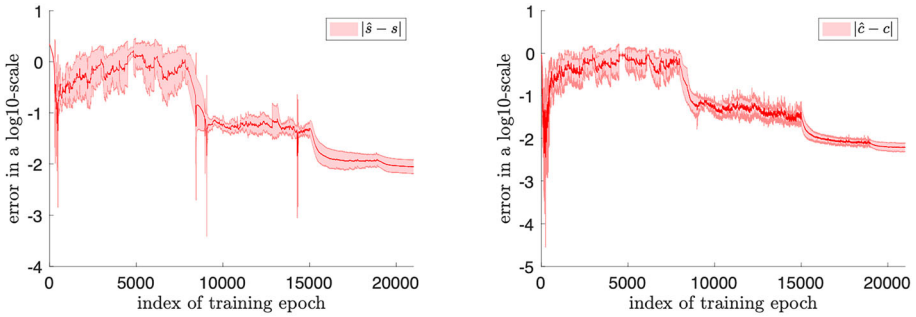

**Fig. 13** Numerical results for the Buckley-Leverett equation (21) with inferred shock fronts ($t_1 = 0.4$, $t_2 = 0.6$)

where $H(x) = \mathbb{1}_{x\geq 0}$ is the Heaviside step function. The Buckley-Leverett equation (21) can then be expressed in terms of our solution ansatz $\hat{u}(x,t,\varphi(x,t))$ as

$$\begin{cases} \partial_t \hat{u}(x,t,\varphi(x,t)) + \partial_x \left( \dfrac{\hat{u}^2(x,t,\varphi(x,t))}{\hat{u}^2(x,t,\varphi(x,t)) + \alpha(1-\hat{u}(x,t,\varphi(x,t)))^2} \right) = 0, & \text{for } (x,t) \in \Omega \setminus \Gamma, \\ \dfrac{f(\hat{u}(x,t,\varphi(x,t)))}{\hat{u}(x,t,\varphi(x,t))} = \hat{s} = f'(\hat{u}(x,t,\varphi(x,t))), & \text{for } (x,t) \in \Gamma = \{x = \hat{s}t\}, \\ \hat{u}(x,0,\varphi(x,0)) = u_0(x), & \text{for } x \in [-1,1], \\ \hat{u}(0,t,\varphi(0,t)) = u(0,t), & \text{for } t \in (0,1], \end{cases} \tag{22}$$

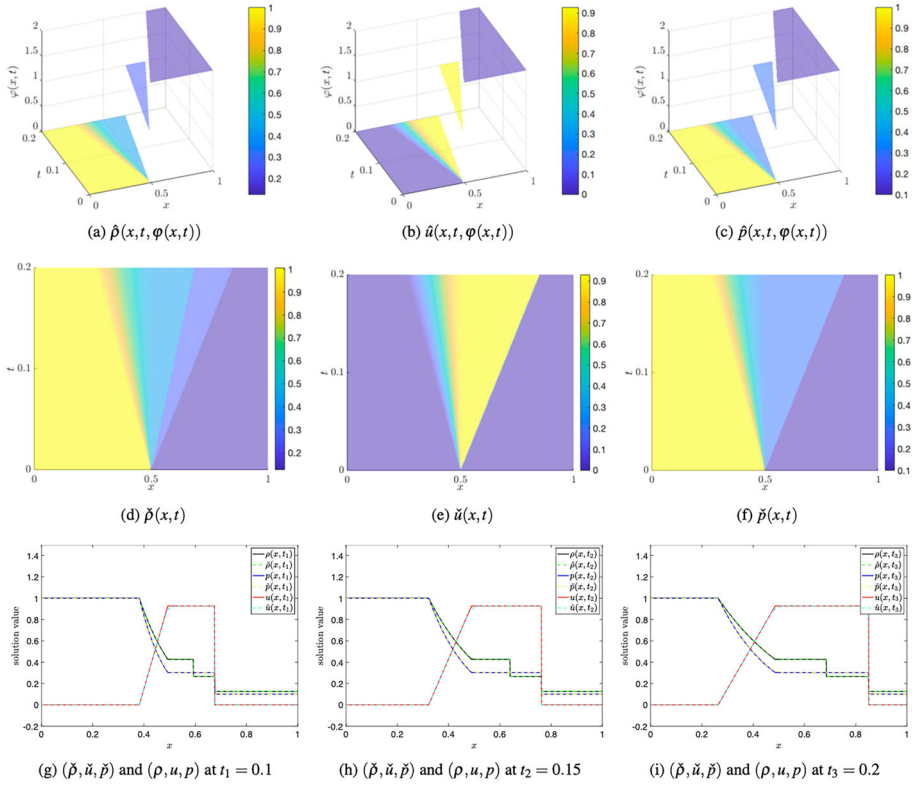
which is solved numerically using the lift-and-embed learning approach proposed in Algorithm 2.

In our numerical experiment, the shock speed is initially set to zero and Figure 13c demonstrates its tendency to approach the true value $s = \frac{\sqrt{3}}{3}$ with satisfactory accuracy across 6 independent runs. As a direct consequence, the numerical solutions depicted in Figure 13e and Figure 13h closely match their analytic counterparts, sharply resolving the shock front and effectively capturing the rarefaction wave in an entropy-admissible manner.




(a) $\log_{10}^{|\hat{s}-s|}$ (mean $\pm$ deviation) (b) $\log_{10}^{|\hat{c}-c|}$ (mean $\pm$ deviation)

**Fig. 14** Numerical propagation speeds of shock wave and contact discontinuity for the Euler equations (23)



(a) $\hat{\rho}(x,t,\varphi(x,t))$ (b) $\hat{u}(x,t,\varphi(x,t))$ (c) $\hat{p}(x,t,\varphi(x,t))$

(d) $\check{\rho}(x,t)$ (e) $\check{u}(x,t)$ (f) $\check{p}(x,t)$

(g) $(\check{\rho},\check{u},\check{p})$ and $(\rho,u,p)$ at $t_1=0.1$ (h) $(\check{\rho},\check{u},\check{p})$ and $(\rho,u,p)$ at $t_2=0.15$ (i) $(\check{\rho},\check{u},\check{p})$ and $(\rho,u,p)$ at $t_3=0.2$

**Fig. 15** Numerical results of the Euler equations (23) for the Sod shock tube problem

### 4.5 Euler Equations with Unknown Discontinuity Locations

Finally, we extend our study from scalar problems to the system of hyperbolic conservation laws, with a focus on the one-dimensional Euler equations of gas dynamics [32, 43], namely,

$$\partial_t \begin{bmatrix} \rho \\ \rho u \\ E \end{bmatrix} + \partial_x \begin{bmatrix} \rho u \\ \rho u^2 + p \\ u(E+p) \end{bmatrix} = 0, \quad \text{or, equivalently,} \quad \partial_t \begin{bmatrix} \rho \\ u \\ p \end{bmatrix} + \begin{bmatrix} u & \rho & 0 \\ 0 & u & 1/\rho \\ 0 & \gamma p & u \end{bmatrix} \partial_x \begin{bmatrix} \rho \\ u \\ p \end{bmatrix} = 0, \tag{23}$$

where $\rho(x,t)$, $u(x,t)$, $p(x,t)$, and $E(x,t)$ denote the density, velocity, pressure, and total energy, respectively. The equations are closed by the ideal gas law

$$p = (\gamma - 1)\left(E - \frac{1}{2}\rho u^2\right),$$

where the constant $\gamma = 1.4$ as is typical for atmospheric gases [32]. More specifically, we consider the Sod shock tube problem [32], with $(x,t) \in [0,1] \times (0, 0.25]$ and the discontinuous initial conditions defined by

$$\rho(x,0) = \begin{cases} 1, & \text{for } x \in [0, 0.5), \\ 0.125, & \text{for } x \in [0.5, 1], \end{cases} \quad p(x,0) = \begin{cases} 1, & \text{for } x \in [0, 0.5), \\ 0.1, & \text{for } x \in [0.5, 1], \end{cases} \quad \text{and} \quad u(x,0) = 0, \quad \text{for } x \in [0,1].$$

It then evolves into a left-moving rarefaction wave, a right-propagating shock wave with speed $s \in \mathbb{R}$ determined by the Rankine-Hugoniot jump conditions

$$s[\![\rho]\!] = [\![\rho u]\!], \quad s[\![\rho u]\!] = [\![\rho u^2 + p]\!], \quad \text{and} \quad s[\![E]\!] = [\![u(E+p)]\!], \\ \text{on } \Gamma_s = \{(x(t), t) \,|\, x(t) = 0.5 + st\},$$

and a slower right-going contact discontinuity $\Gamma_c = \{(x(t), t) \,|\, x(t) = 0.5 + ct\}$ at a local fluid velocity $c \in \mathbb{R}$ [43], across which the density exhibits a jump while the velocity and pressure remain continuous.

Therefore, our augmented variable is constructed as

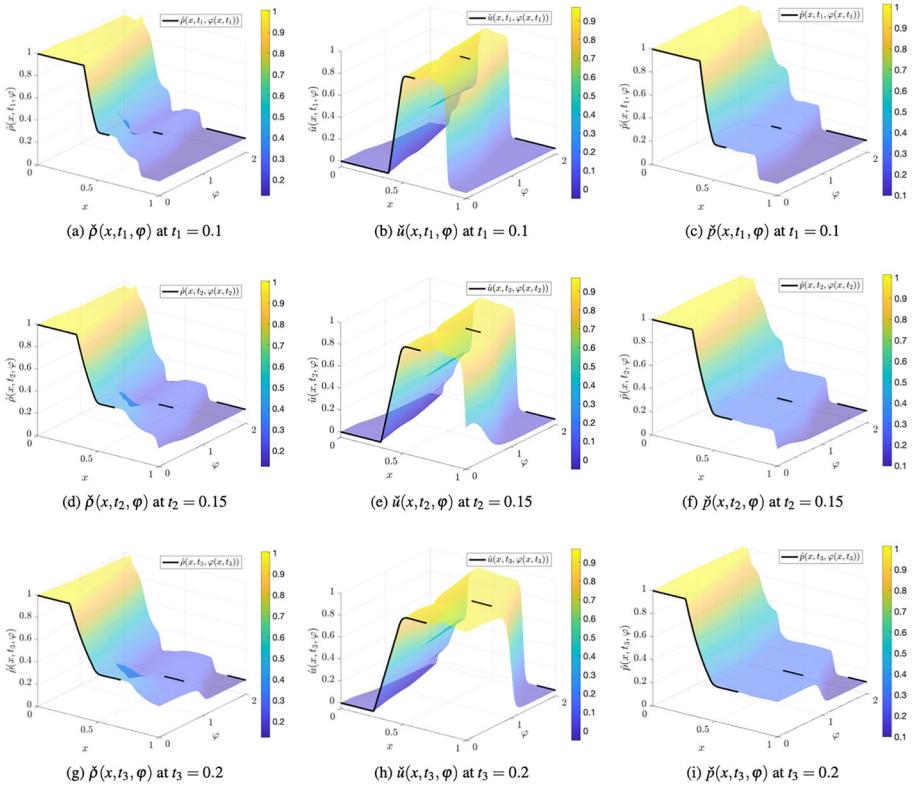
$$\varphi(x,t) = H(x - \hat{s}t - 0.5) + H(x - \hat{c}t - 0.5), \tag{24}$$

where $\hat{s} \in \mathbb{R}$ and $\hat{c} \in \mathbb{R}$ indicate the approximate propagation speeds of shock wave and contact discontinuity. On the other hand, one single fully-connected neural network [29] with three output neurons are applied to represent our solution ansatz $\hat{\rho}(x,t,\varphi(x,t))$, $\hat{u}(x,t,\varphi(x,t))$, and $\hat{p}(x,t,\varphi(x,t))$, respectively, leading to the reformulation of Euler equations (23) in terms of primitive variables as (variables $(x,t,\varphi(x,t))$ are omitted for brevity)

$$\begin{cases} \partial_t \begin{bmatrix} \hat{\rho} \\ \hat{u} \\ \hat{p} \end{bmatrix} + \begin{bmatrix} \hat{u} & \hat{\rho} & 0 \\ 0 & \hat{u} & 1/\hat{\rho} \\ 0 & \gamma\hat{p} & \hat{u} \end{bmatrix} \partial_x \begin{bmatrix} \hat{\rho} \\ \hat{u} \\ \hat{p} \end{bmatrix} = 0, & \text{for } (x,t) \in \Omega \setminus \{\Gamma_{\hat{s}} \cup \Gamma_{\hat{c}}\}, \\ \hat{s}[\![\hat{u}]\!] = [\![\hat{\rho}\hat{u}]\!], \quad \hat{s}[\![\hat{\rho}\hat{u}]\!] = [\![\hat{\rho}\hat{u}^2 + \hat{p}]\!], \quad \hat{s}\left[\!\!\left[\frac{1}{2}\hat{\rho}\hat{u}^2 + \frac{\hat{p}}{\gamma - 1}\right]\!\!\right] = \left[\!\!\left[\hat{u}\left(\frac{1}{2}\hat{\rho}\hat{u}^2 + \frac{\gamma\hat{p}}{\gamma - 1}\right)\right]\!\!\right], & \text{for } (x,t) \in \Gamma_{\hat{s}}, \\ \hat{c} = \hat{u}, \quad [\![\hat{u}]\!] = [\![\hat{p}]\!] = 0, & \text{for } (x,t) \in \Gamma_{\hat{c}}, \\ \hat{\rho} = \rho(x,0), \quad \hat{u} = u(x,0), \quad \hat{p} = p(x,0), & \text{for } x \in [0,1] \text{ and } t = 0. \end{cases}$$

Notably, while velocity and pressure are continuous across the contact discontinuity $\Gamma_{\hat{c}}$, their augmented variable may introduce discontinuities due to the Heaviside function in (24), zero jump constraints $[\![\hat{u}]\!] = [\![\hat{p}]\!] = 0$ are then included to enforce continuity.

By applying a single fully-connected neural network with three output neurons for the density $\hat{\rho}(x,t,\varphi(x,t))$, velocity $\hat{u}(x,t,\varphi(x,t))$ and pressure $\hat{p}(x,t,\varphi(x,t))$, both the solution ansatz and unknown parameters $\hat{s}, \hat{c} \in \mathbb{R}$ can be optimized concurrently within our unified learning framework (Algorithm 2). More specifically, we perform 6 independent runs with initial parameter values $\hat{s} = 4$ and $\hat{c} = 0$, achieving reliable agreement with their reference values [43, 61] as displayed in Figure 14.



(a) $\check{\rho}(x,t_1,\varphi)$ at $t_1 = 0.1$ (b) $\check{u}(x,t_1,\varphi)$ at $t_1 = 0.1$ (c) $\check{p}(x,t_1,\varphi)$ at $t_1 = 0.1$

(d) $\check{\rho}(x,t_2,\varphi)$ at $t_2 = 0.15$ (e) $\check{u}(x,t_2,\varphi)$ at $t_2 = 0.15$ (f) $\check{p}(x,t_2,\varphi)$ at $t_2 = 0.15$

(g) $\check{\rho}(x,t_3,\varphi)$ at $t_3 = 0.2$ (h) $\check{u}(x,t_3,\varphi)$ at $t_3 = 0.2$ (i) $\check{p}(x,t_3,\varphi)$ at $t_3 = 0.2$

**Fig. 16** Snapshots of the Euler equations embedded in a higher-dimensional space (corresponds to Figure 15)

Accordingly, our trained solution ansatz lie in a one-order higher-dimensional space, as shown in Figure 15a, Figure 15b, and Figure 15c, while their projections onto the spatial-temporal domain are displayed in Figure 15d, Figure 15e, and Figure 15f. Furthermore, Figure 15g, Figure 15h, and Figure 15i display snapshots of our trained density, velocity, and pressure at different temporal slices, illustrating good agreement with reference solutions in capturing both the shock wave and contact discontinuity [43]. This is achieved through embedding discontinuous snapshots into smooth hyperplanes in a higher-dimensional space (Figure 16), allowing standard neural networks to reconstruct discontinuous solutions effectively.

Notably, the approximation of rarefaction waves at the leading edge (Figure 15g, Figure 15h, and Figure 15i) still exhibits observable errors due to the derivative discontinuities, in which our smooth neural networks struggle to reconstruct accurately. Designing proper augmented variables to address this limitation provides an interesting direction for our future research.

## 5 Conclusions

In this paper, we present a lift-and-embed learning method for solving scalar hyperbolic equations with discontinuous solutions, accompanied by extensive numerical experiments

that validate the effectiveness and flexibility of our method. By including an augmented variable to embed our solution ansatz into a one-order higher-dimensional space, the Rankine-Hugoniot jump condition is expressed at separate collocation points, enabling not only the effective management of multivalued functions but also the use of smooth neural networks to reconstruct discontinuous solutions. Moreover, our method facilitate the numerical solution of both linear and nonlinear problems within a unified learning framework, accommodating scenarios with known and unknown locations of discontinuities. It is also noteworthy that our modified equations are defined on piecewise surfaces within the elevated-dimensional space, rather than fulfilling the entire domain, hence the sampling of collocation points remains unchanged despite the increased dimensionality. Experimental studies on various problems are reported, which numerically demonstrate that our proposed method is capable of resolving discontinuities without spurious smearing and oscillations.

For future work, we anticipate that our framework could be extended to more complex systems of hyperbolic conservation laws beyond the one-dimensional Euler equations [28]. Additionally, conducting a comprehensive error analysis is of critical importance and will be a key focus of future investigations. It is also noteworthy that, with recent advancements in physics-informed operator learning [44, 45, 50, 59], applying our lift-and-embed method to infer solution operators for hyperbolic problems offers a compelling research direction.

**Acknowledgements** Q. Sun is supported in part by National Natural Science Foundation of China (grant 12201465), Science and Technology Commission of Shanghai Municipality (grant 23JC1400502), and Shanghai Municipal Science and Technology Major Project (No. 2021SHZDZX0100). X. Xu is supported in part by National Natural Science Foundation of China (grants 12071350 and 12331015). This research was conducted using the computational resources and services of the HPC Center at the School of Mathematical Sciences, Tongji University.

**Data Availability** Enquiries about data availability should be directed to the authors. All source codes, along with detailed hyperparameter configurations, are publicly accessible on GitHub (https://github.com/q1sun/LELM_HyperbolicPDE).

## Declarations

**Conflict of interest** The authors have no conflict of interest.

## References

1. Arora, R.: A deep learning framework for solving hyperbolic partial differential equations: Part I. arXiv preprint arXiv:2307.04121 (2023)
2. Bar-Sinai, Y., Hoyer, S., Hickey, J., Brenner, M.P.: Learning data-driven discretizations for partial differential equations. Proc. Natl. Acad. Sci. **116**(31), 15344–15349 (2019)
3. Barron, A.R.: Universal approximation bounds for superpositions of a sigmoidal function. IEEE Trans. Inf. Theory **39**(3), 930–945 (1993)
4. Bois, L., Franck, E., Navoret, L., Vigon, V.: An optimal control deep learning method to design artificial viscosities for discontinuous Galerkin schemes. J. Sci. Comput. **101**(3), 70 (2024)
5. Braga-Neto, U.: Characteristics-informed neural networks for forward and inverse hyperbolic problems. arXiv preprint arXiv:2212.14012 (2022)
6. Buckley, S.E., Leverett, M.: Mechanism of fluid displacement in sands. Trans. AIME **146**(01), 107–116 (1942)
7. Cai, Z., Chen, J., Liu, M.: Least-squares ReLU neural network (LSNN) method for scalar nonlinear hyperbolic conservation law. Appl. Numer. Math. **174**, 163–176 (2022)
8. Cai, Z., Chen, J., Liu, M.: Least-squares neural network (LSNN) method for scalar nonlinear hyperbolic conservation laws: Discrete divergence operator. J. Comput. Appl. Math. **433**, 115298 (2023)
9. Cai, Z., Hejnal, B.: Evolving neural network (ENN) method for one-dimensional scalar hyperbolic conservation laws: I linear and quadratic fluxes. arXiv preprint arXiv:2312.06919 (2023)

10. Cen, J., Zou, Q.: Deep finite volume method for partial differential equations. J. Comput. Phys. **517**, 113307 (2024)
11. Chen, C., Yang, Y., Xiang, Y., Hao, W.: Learn singularly perturbed solutions via homotopy dynamics. arXiv preprint arXiv:2502.00488 (2025)
12. Chen, J., Jin, S., Lyu, L.: A deep learning based discontinuous Galerkin method for hyperbolic equations with discontinuous solutions and random uncertainties. arXiv preprint arXiv:2107.01127 (2021)
13. Cockburn, B., Karniadakis, G.E., Shu, C.W.: Discontinuous Galerkin Methods: Theory, Computation and Applications, vol. 11. Springer Science & Business Media (2012)
14. Coutinho, E.J.R., Dall'Aqua, M., McClenny, L., Zhong, M., Braga-Neto, U., Gildin, E.: Physics-informed neural networks with adaptive localized artificial viscosity. J. Comput. Phys. **489**, 112265 (2023)
15. Cuomo, S., Di Cola, V.S., Giampaolo, F., Rozza, G., Raissi, M., Piccialli, F.: Scientific machine learning through physics–informed neural networks: where we are and what's next. J. Sci. Comput. **92**(3), 88 (2022)
16. Dahmen, W., Huang, C., Schwab, C., Welper, G.: Adaptive Petrov-Galerkin methods for first order transport equations. SIAM J. Numer. Anal. **50**(5), 2420–2445 (2012)
17. Daw, A., Bu, J., Wang, S., Perdikaris, P., Karpatne, A.: Mitigating propagation failures in physics-informed neural networks using retain-resample-release (R3) sampling. arXiv preprint arXiv:2207.02338 (2022)
18. De Ryck, T., Mishra, S., Molinaro, R.: wPINNs: Weak physics informed neural networks for approximating entropy solutions of hyperbolic conservation laws. SIAM J. Numer. Anal. **62**(2), 811–841 (2024)
19. Della Santa, F., Pieraccini, S.: Discontinuous neural networks and discontinuity learning. J. Comput. Appl. Math. **419**, 114678 (2023)
20. Demkowicz, L., Gopalakrishnan, J.: A class of discontinuous Petrov–Galerkin methods. Part I: The transport equation. Comput. Methods Appl. Mech. and Engrg. **199**(23-24), 1558–1572 (2010)
21. Di, X., Shi, R., Mo, Z., Fu, Y.: Physics-informed deep learning for traffic state estimation: a survey and the outlook. Algorithms **16**(6), 305 (2023)
22. Diab, W., Kobaisi, M.A.: PINNs for the solution of the hyperbolic Buckley-Leverett problem with a non-convex flux function. arXiv preprint arXiv:2112.14826 (2021)
23. Evans, L.C.: Partial Differential Equations, vol. 19. American Mathematical Society (2022)
24. Feng, X., Shangguan, H., Tang, T., Wan, X., Zhou, T.: A hybrid FEM-PINN method for time-dependent partial differential equations. arXiv preprint arXiv:2409.02810 (2024)
25. Fuks, O., Tchelepi, H.A.: Limitations of physics informed machine learning for nonlinear two-phase transport in porous media. J. Mach. Learn. Model. Comput. **1**(1), 19–37 (2020)
26. Gao, Z., Tang, T., Yan, L., Zhou, T.: Failure-informed adaptive sampling for PINNs; Part II: combining with re-sampling and subset simulation. Commun. Appl. Math. Comput **6**(3), 1720–1741 (2024)
27. Gao, Z., Yan, L., Zhou, T.: Failure-informed adaptive sampling for PINNs. SIAM J. Sci. Comput. **45**(4), A1971–A1994 (2023)
28. Godlewski, E., Raviart, P.A.: Numerical Approximation of Hyperbolic Systems of Conservation Laws, vol. 118. Springer Science & Business Media (2013)
29. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016)
30. Gustafsson, B., Kreiss, H.O., Oliger, J.: Time Dependent Problems and Difference Methods, vol. 2. Wiley Online Library (2013)
31. Harten, A., Engquist, B., Osher, S., Chakravarthy, S.R.: Uniformly high order accurate essentially non-oscillatory schemes III. J. Comput. Phys. **131**(1), 3–47 (1997)
32. Hesthaven, J.S.: Numerical Methods for Conservation Laws: From Analysis to Algorithms. SIAM (2017)
33. Hornik, K., Stinchcombe, M., White, H.: Multilayer feedforward networks are universal approximators. Neural Netw. **2**(5), 359–366 (1989)
34. Hu, W.F., Lin, T.S., Lai, M.C.: A discontinuity capturing shallow neural network for elliptic interface problems. J. Comput. Phys. **469**, 111576 (2022)
35. Huang, A.J., Agarwal, S.: On the limitations of physics-informed deep learning: Illustrations using first-order hyperbolic conservation law-based traffic flow models. IEEE Open J. Intell. Transp. Syst. **4**, 279–293 (2023)
36. Jagtap, A.D., Kawaguchi, K., Karniadakis, G.E.: Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. J. Comput. Phys. **404**, 109136 (2020)
37. Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Yang, L.: Physics-informed machine learning. Nat. Rev. Phys. **3**(6), 422–440 (2021)
38. Kharazmi, E., Zhang, Z., Karniadakis, G.E.: Variational physics-informed neural networks for solving partial differential equations. arXiv preprint arXiv:1912.00873 (2019)
39. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)
40. Kitamura, K., Shima, E., Roe, P.L.: Carbuncle phenomena and other shock anomalies in three dimensions. AIAA J. **50**(12), 2655–2669 (2012)

41. Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., Mahoney, M.W.: Characterizing possible failure modes in physics-informed neural networks. Adv. Neural. Inf. Process. Syst. **34**, 26548–26560 (2021)
42. Lagaris, I.E., Likas, A., Fotiadis, D.I.: Artificial neural networks for solving ordinary and partial differential equations. IEEE Trans. Neural Netw. **9**(5), 987–1000 (1998)
43. LeVeque, R.J.: Finite Volume Methods for Hyperbolic Problems, vol. 31. Cambridge University Press (2002)
44. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A.: Fourier neural operator for parametric partial differential equations. arXiv preprint arXiv:2010.08895 (2020)
45. Lin, B., Mao, Z., Wang, Z., Karniadakis, G.E.: Operator learning enhanced physics-informed neural networks for solving partial differential equations characterized by sharp solutions. arXiv preprint arXiv:2310.19590 (2023)
46. Liu, L., Liu, S., Xie, H., Xiong, F., Yu, T., Xiao, M., Liu, L., Yong, H.: Discontinuity computing using physics-informed neural networks. J. Sci. Comput. **98**(1), 22 (2024)
47. Liu, W., Yan, L., Zhou, T., Zhou, Y.: Failure-informed adaptive sampling for PINNs; Part III: applications to inverse problems. CSIAM Trans. Appl. Math **5**(3), 636–670 (2024)
48. Lorin, E., Novruzi, A.: Non-diffusive neural network method for hyperbolic conservation laws. J. Comput. Phys. p. 113161 (2024)
49. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101 (2017)
50. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E.: Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nat. Mach. Intell. **3**(3), 218–229 (2021)
51. Mao, Z., Jagtap, A.D., Karniadakis, G.E.: Physics-informed neural networks for high-speed flows. Comput. Methods Appl. Mech. Eng. **360**, 112789 (2020)
52. Mojgani, R., Balajewicz, M., Hassanzadeh, P.: Kolmogorov n-width and Lagrangian physics-informed neural networks: a causality-conforming manifold for convection-dominated PDEs. Comput. Methods Appl. Mech. Eng. **404**, 115810 (2023)
53. Morand, V., Müller, N., Weightman, R., Piccoli, B., Keimer, A., Bayen, A.M.: Deep learning of first-order nonlinear hyperbolic conservation law solvers. J. Comput. Phys. **511**, 113114 (2024)
54. Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., Lerer, A.: Automatic differentiation in PyTorch (2017)
55. Patel, R.G., Manickam, I., Trask, N.A., Wood, M.A., Lee, M., Tomas, I., Cyr, E.C.: Thermodynamically consistent physics-informed neural networks for hyperbolic systems. J. Comput. Phys. **449**, 110754 (2022)
56. Peery, K., Imlay, S.: Blunt-body flow simulations. In: 24th joint propulsion conference, p. 2904 (1988)
57. Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J. Comput. Phys. **378**, 686–707 (2019)
58. Rodriguez-Torrado, R., Ruiz, P., Cueto-Felgueroso, L., Green, M.C., Friesen, T., Matringe, S., Togelius, J.: Physics-informed attention-based neural network for hyperbolic partial differential equations: application to the Buckley-Leverett problem. Sci. Rep. **12**(1), 7557 (2022)
59. Rosofsky, S.G., Al Majed, H., Huerta, E.: Applications of physics informed neural operators. Mach. Learn.: Sci. Technol. **4**(2), 025022 (2023)
60. Shu, C.W., Osher, S.: Efficient implementation of essentially non-oscillatory shock-capturing schemes. J. Comput. Appl. Math. **77**(2), 439–471 (1988)
61. Sod, G.A.: A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws. J. Comput. Phys. **27**(1), 1–31 (1978)
62. Süli, E., Mayers, D.F.: An Introduction to Numerical Analysis. Cambridge University Press (2003)
63. Sun, Z.: Convolution neural network shock detector for numerical solution of conservation laws. Commu. Comput. Phys. **28**(5) (2020)
64. Tseng, Y.H., Lin, T.S., Hu, W.F., Lai, M.C.: A cusp-capturing PINN for elliptic interface problems. J. Comput. Phys. **491**, 112359 (2023)
65. Wang, T., Zhao, B., Gao, S., Yu, R.: Understanding the difficulty of solving Cauchy problems with PINNs. arXiv preprint arXiv:2405.02561 (2024)
66. Xu, J.: The finite neuron method and convergence analysis. arXiv preprint arXiv:2010.01458 (2020)
67. Zhang, X., Cheng, T., Ju, L.: Implicit form neural network for learning scalar hyperbolic conservation laws. In: Mathematical and Scientific Machine Learning, pp. 1082–1098. PMLR (2022)
68. Zhou, N., Ma, Z.: Capturing shock waves by relaxation neural networks. arXiv preprint arXiv:2404.01163 (2024)


**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.